

Cartridge & Cloud — Economy and Balance Specification

Economía, inventario, precios, demanda, ciclo diario, tuning y validación

VRM Games / Blas Luis Rocha González

2026-07-06

Cartridge & Cloud — Economy and Balance Specification

Archivo: 20_Economy_and_Balance_Specification.md

Propósito: definir la autoridad económica y el procedimiento de diseño, implementación, tuning, prueba y aprobación de los sistemas de dinero, inventario, proveedores, pedidos, clientes, checkout, ciclo diario y progresión de Cartridge & Cloud.

Estado del proyecto: Sprints 0–15 CLOSED / PASS; Sprint 16 COMPLETED / PASS; Sprint 17 PENDING / READY TO OPEN; H6 BLOCKED / NOT RUN.

Estado económico observado: núcleo transaccional implementado, balance final pendiente; dos capas de catálogo coexistentes; campaña económica de siete días aún no aprobada en build externa.

Regla de interpretación: una fórmula correcta y tests verdes demuestran integridad, no diversión, ritmo, claridad ni equilibrio. Los valores históricos se preservan, pero solo son vigentes cuando la jerarquía y la evidencia actual los ratifican.

Esta especificación incorpora todas las versiones históricas encontradas: Economy & Balance v0.1 de las baselines v0.3 y v0.4; v0.2 de v0.5; v0.3 de v0.6; GDD v0.4, v0.5 y v0.6; Initial Content Catalog v0.1, v0.2 y v0.3; ADR, planes, matrices, cierres y handoffs de Sprints 6–16; código, tests, ScriptableObjects, catálogos y snapshots reales; y los documentos consolidados 00–19. Ninguna versión previa se descarta: se clasifica como origen conceptual, regla vigente, decisión sustituida, dato representativo, deuda o visión futura. # 0. Control documental Este documento ocupa la posición 20 del paquete consolidado. Su autoridad cubre reglas económicas, fórmulas, parámetros, invariantes, datos de balance, métricas, escenarios

y criterios de aprobación. No sustituye al GDD para la fantasía, al TDD para arquitectura, al Modelo de Datos para serialización, al QA Plan para política de pruebas ni al catálogo de contenido para el inventario editorial. Cuando define un número, debe indicar si es histórico, técnico, runtime, objetivo de tuning o futuro. - **DEBE**: requisito obligatorio. - **NO DEBE**: práctica prohibida salvo excepción formal. - **DEBERÍA**: recomendación fuerte. - **PUEDE**: opción permitida. - **HISTÓRICO**: valor preservado, no necesariamente activo. - **RUNTIME**: valor consumido por el flujo jugable actual. - **TÉCNICO**: asset o escenario de validación de un subsistema. - **OBJETIVO DE TUNING**: rango que requiere campaña y evidencia. - **VISIÓN**: sistema futuro no abierto. La revisión del documento es obligatoria cuando cambien precios, costes, capacidades, capital inicial, duración del día, perfiles de cliente, reglas de checkout, momento de reconocimiento de costes, schemas, dificultad, métricas o gates. Un ajuste de balance sin registro de versión y evidencia se considera cambio no controlado.

1. Propósito y resultados esperados La especificación convierte reglas dispersas en un modelo verificable que permita responder cuánto cuesta operar, qué decisiones enfrenta el jugador, qué resultados son saludables, cómo se detecta una estrategia dominante y cuándo un cambio numérico mejora o empeora la experiencia. El objetivo no es producir una hoja de cálculo aislada, sino conectar cada cifra con comportamiento, UI, persistencia, pruebas y trazabilidad. 1. preservar integridad monetaria e inventario; 2. hacer legibles coste, precio, margen y consecuencia; 3. evitar pérdidas, duplicaciones y cobros dobles; 4. mantener decisiones de stock y espacio relevantes; 5. permitir más de una estrategia razonable; 6. evitar bancarrota inevitable y softlocks tempranos; 7. proporcionar datos ajustables sin contaminar dominio; 8. definir métricas y experimentos de Sprint 17; 9. establecer evidencia económica para H6; 10. conservar la visión futura sin abrir alcance.

2. Jerarquía de autoridad 1. 00_Enfoque_y_Alcance.md para promesa, límites y fases. 2. 01_Game_Design_Document.md para bucle, fantasía y sistemas comprometidos. 3. 02_Vertical_Slice_Specification.md para aceptación H6. 4. 03_Technical_Design_Document.md y 04_Modelo_de_Datos.md para contratos, atomicidad y persistencia. 5. Este documento para reglas, fórmulas, parámetros, tuning y métricas. 6. 05_UX_Flow.md y 19_UI_Style_Guide.md para presentación y decisiones. 7. 07 y 08 para ejecución QA. 8. 12 y 13 para estado y control de cambios. 9. Assets y código para evidencia de implementación. 10. Fuentes históricas para genealogía cuando no contradicen una decisión vigente. Si el GDD histórico indica capital inicial de 20.000 € pero CC_Sprint15Settings.asset inicializa 1.000 €, el primero se conserva como intención histórica y el segundo es el valor runtime observado. La discrepancia no se resuelve

ocultándola: Sprint 17 debe escoger, justificar y sincronizar datos, documentación, pruebas y save. # 3. Alcance y exclusiones Incluye moneda, caja, precios, costes, márgenes, inventario, proveedores, pedidos, recepción, displays, demanda, perfiles, reservas, carrito, cola, checkout, ledger, día, métricas, dificultad, tuning, simulación y progresión económica. Incluye visión futura solo como frontera y principios. Excluye contratos legales reales, fiscalidad real, contabilidad reglada, monetización externa y microtransacciones. El vertical slice exige una economía pequeña pero completa: comprar, recibir, colocar, vender, cerrar, guardar, cargar y continuar. Empleados, investigación, ecommerce, publishing, desarrollo interno y plataforma no deben introducir costes o ingresos activos antes de sus gates. # 4. Vocabulario de estado económico | Estado | Significado | | — | — | | Conceptual | regla de diseño sin implementación | | Implementado | código o asset existe | | Integrado | participa en el flujo runtime | | Verificado | tests o inspección confirman contrato | | Balanceado | campaña demuestra rangos aceptables | | Aprobado | gate formal con build y evidencia | | Placeholder | valor temporal para flujo | | Diferido | fuera de la fase actual | | Sustituido | conservado solo por historia |

No se usará “cerrado” para un valor que solo compila. El núcleo económico de Sprint 13 está implementado y probado; los valores de campaña siguen pendientes de balance. La distinción protege el proyecto frente a conclusiones prematuras. # 5. Inventario de fuentes históricas Se han extraído 92 documentos Markdown de economía, GDD, catálogos, ADR, aceptación, tests, cierres y handoffs. La genealogía incluye duplicados controlados entre baselines porque una reedición puede cambiar su regla de interpretación aunque conserve texto. El apéndice de fuentes registra nombre, procedencia, palabras y SHA-256. | Línea | Versiones | Uso | | — | — | — | | Economy & Balance | v0.1 (v0.3/v0.4), v0.2, v0.3 | visión inicial → mínimo técnico → estado implementado | | GDD | v0.4, v0.5, v0.6 | fantasía, alcance y evolución | | Content Catalog | v0.1, v0.2, v0.3 | familias conceptuales → contenido representativo | | ADR | 0024–0073 relevantes | decisiones técnicas | | Sprints | 6–15 y S16 Phase 1 | evidencia de implementación y gates |

6. Economy & Balance v0.1 — visión extensa

La versión v0.1 definió una economía empresarial completa desde tienda pequeña hasta plataforma digital. Aportó capital estándar de 20.000 €, inversión de apertura de 13.000–15.000 €, costes fijos de 200–215 €/día, impuesto semanal del 10 %, objetivo de margen bruto del 35 %, ventas saludables de 800–1.200 €/día, dificultad y múltiples expansiones. También describió empleados, eventos, investigación, ecommerce y servicios informáticos. Su valor actual es doble: conserva intención de largo plazo y proporciona hipótesis de tuning. No es el run-

time vigente del vertical slice. Muchos números fueron diseñados antes de que existieran productos, código, tiempos y flujo real; deben tratarse como rangos históricos que pueden ser recuperados, ajustados o sustituidos mediante evidencia. # 7. Economy & Balance v0.2 — reducción al mínimo La versión v0.2, publicada tras Sprint 5, abandonó el exceso de cifras y fijó principios: coste por producto/proveedor, precio editable dentro de límites, impuesto y gasto fijo configurables, beneficio como ingresos menos coste vendido y gastos, y telemetría de ventas, roturas de stock, abandono, cola, displays y margen. Declaró explícitamente que la economía todavía no estaba implementada. Esta reducción fue correcta para proteger la arquitectura. Muchas fórmulas siguen vigentes como objetivo, pero el runtime actual aún no implementa impuesto, alquiler, servicios, salarios ni coste vendido contable. El resultado diario actual es ingreso de checkout menos coste de recepciones del día, una métrica distinta de beneficio neto. # 8. Economy & Balance v0.3 — núcleo implementado La versión v0.3 reconoce el cierre de Sprints 0–15 y establece reglas ya implementadas: dinero en unidades menores enteras, moneda compatible, coste de proveedor autoritativo, precio de venta en catálogo separado, quote antes de checkout, coherencia entre venta, stock y ledger, y balance final reservado a Sprint 17. Enumera capital, seis productos, mobiliario, entregas, capacidades, clientes, paciencia, duración del día y progresión como pendientes de revisión. Esta especificación amplía v0.3 con inspección de código y datos. Confirma el núcleo, pero detecta dos capas de contenido con valores distintos y una implementación Phase 1 adicional que muta caja directamente. El trabajo de Sprint 17 debe unificar esas capas antes de considerar los números definitivos. # 9. Evolución del GDD y de los catálogos Los GDD v0.4 y catálogos v0.1 describieron una visión amplia: precios ajustables 50–150 %, promociones, proveedores especializados, reservas, devoluciones, robos, puestos informáticos, empleados y expansiones. V0.5 redujo el contenido a familias representativas. V0.6 fijó seis productos funcionales y mobiliario para Sprint 16. La documentación actual conserva la visión, pero limita el compromiso a la tienda física y al vertical slice. Los conceptos históricos no se eliminan porque ayudan a evitar callejones sin salida. Sin embargo, “aprobado conceptualmente” no significa “activo”. Cada sistema futuro necesitará requisito, modelo de datos, UX, balance, QA y gate propios. # 10. Estado económico real observado El proyecto dispone de moneda EUR, caja persistida, pedidos, recepción, inventarios, displays, reservas, checkout, ledger, resumen diario y save integrado. El runtime principal inicializa 1.000.00 € y un día de 300 segundos. El catálogo Phase 1 contiene 6 productos y 8 muebles. Los assets técnicos contienen 6 productos, 2 proveedores, 4 perfiles y 3 definiciones de display. No existe todavía economía completa de lanzamiento: no hay alquiler, electricidad, salarios, impuestos, promociones, precio editable, presupuesto de cliente, sensibilidad al precio, deterioro, devoluciones, robo, préstamo o investigación. El ledger técnico solo reconoce **CheckoutRevenue** y **SupplierReceivingCost**. La implementación Phase 1 sí mantiene caja, ingresos y gastos acumulados, pero no calcula beneficio neto con costes fijos. # 11. Dos capas económicas coexistentes | Capa | Autoridad actual | Contenido | Riesgo | | — | — | — | — | | Dominio S6–S14 | contratos e invariantes | ProductCatalog técnico, dos sup-

pliers, PriceCatalog, Money, Ledger, checkout | no es el catálogo referenciado por RuntimeAssetRegistry Phase 1 | | Phase 1/S16 | flujo jugable representativo | ContentCatalog, caja directa, muebles, seis productos, órdenes y ventas | duplica IDs y precios; lógica simplificada |

RuntimeAssetRegistry.asset referencia ContentCatalog.asset, no CC_ProductCatalog_Technical.asset. Por tanto, los valores Phase 1 son los consumidos por el flujo representativo. Los assets S13 siguen siendo evidencia válida del dominio y sus tests. Sprint 17 debe elegir una autoridad única o definir una proyección explícita entre ambas; mantener dos catálogos independientes produciría resultados, saves y pruebas divergentes. # 12. Pilares económicos 1. **Integridad:** nunca perder ni duplicar unidades o dinero. 2. **Legibilidad:** el jugador entiende coste, precio, stock y consecuencia. 3. **Decisión:** comprar, colocar y vender implica trade-offs reales. 4. **Recuperación:** un error razonable no crea un softlock irreversible. 5. **Determinismo auditable:** el mismo estado y acción producen resultado explicable. 6. **Ajustabilidad:** valores en datos, no dispersos por código. 7. **Ritmo:** actividad suficiente sin saturación. 8. **Diversidad:** varias estrategias viables, sin producto dominante absoluto. 9. **Trazabilidad:** todo cambio numérico tiene motivo y evidencia. 10. **Alcance protegido:** sistemas futuros no alteran H6. # 13. Bucle económico del vertical slice

Caja disponible
→ realizar pedido
→ recibir y pagar
→ almacenar
→ colocar mobiliario
→ asignar producto
→ reponer display
→ atraer/resolver cliente
→ reservar y pasar por checkout
→ cobrar
→ cerrar día
→ guardar y continuar

El pedido Phase 1 no inmoviliza caja al colocarse; el pago ocurre al recibir. Esto permite crear pedidos cuyo coste excede la caja futura y fallar en recepción. Es comportamiento implementado, no necesariamente balance final. Si se mantiene, la UI debe mostrar compromiso pendiente y riesgo; si se cambia, debe decidirse entre preautorización, reserva de fondos o pago al pedido. # 14. Fronteras de autoridad entre lógica y balance El dominio define qué estados son válidos; los datos definen cuánto cuestan y cuánto duran. Money, InventoryTransferService, CheckoutService y EconomyLedger no deben conocer dificultad, promociones o curva de siete días. Un balanceador puede modificar precios y capacidades sin reescribir atomicidad. La capa de aplicación coordina la transacción; Presentation explica el resultado. Cuando una cifra cambia una regla —por ejemplo, permitir cantidad cero, vender bajo coste o

mezclar monedas— deja de ser tuning y se convierte en cambio de arquitectura o diseño, posiblemente con ADR. # 15. Unidad monetaria y moneda **Money** almacena **long MinorUnits** y **CurrencyCode**. EUR se serializa con tres letras ASCII mayúsculas. Suma, resta, comparación y multiplicación usan operaciones comprobadas y rechazan monedas incompatibles. Esta regla elimina errores binarios de **float** y mantiene resultados exactos. La UI convierte céntimos a formato localizado; el dominio no incrusta símbolo ni separador. No se redondea durante suma de líneas porque todos los precios son enteros. Cualquier fórmula porcentual futura deberá especificar orden, precisión intermedia y regla de redondeo. # 16. Reglas de redondeo La implementación actual evita porcentajes, por lo que no necesita redondear precios o impuestos. Cuando se introduzcan descuentos, tasas o multiplicadores, la especificación propone trabajar con enteros y fracciones racionales: **resultado = round_div(base * numerador, denominador)**. La política debe ser única por concepto y probada en límites. - Precios mostrados: dos decimales para EUR. - Descuentos: calcular por línea o total según regla explícita, nunca alternar. - Impuestos: redondear una vez al periodo definido. - Promedios: pueden usar decimal para análisis, no para mutar caja. - No ocultar céntimos residuales ni usar tolerancias monetarias. # 17. Formato visual y comunicación Dinero debe mostrarse con símbolo, separador y signo coherentes. Coste, precio y margen no deben confundirse. Un pedido mostrará coste total, unidades por caja, cajas, fecha o estado de recepción y caja disponible. Checkout muestra total confirmado, no una estimación distinta al ledger. El cierre diario diferencia ingresos, costes de recepción y resultado bruto. Los valores históricos en euros no deben aparecer en UI de producción mientras no sean runtime. El texto debe indicar “representativo” o quedar fuera del jugador. La localización ES/EN deberá probar separadores, longitudes y pluralización. # 18. Caja, ingresos, costes y resultado La caja del snapshot es el saldo operativo disponible. Las ventas la incrementan y la recepción la reduce. **LifetimeRevenueCents** y **LifetimeExpenseCents** son acumulados Phase 1. El ledger conserva postings por fuente y día. El **DailyEconomicResult** técnico calcula **GrossResult = CheckoutRevenue - SupplierReceivingCost**. Ninguno de estos conceptos equivale automáticamente a beneficio neto contable. La especificación reserva “beneficio neto” para ingresos menos coste vendido, gastos, impuestos y ajustes del periodo. Mientras esos componentes no existan, la UI y documentación deben usar “resultado bruto técnico” o “flujo neto de caja registrado”. # 19. Invariantes contables mínimas - caja nunca negativa después de una operación confirmada; - cada ingreso de checkout tiene una transacción completada única; - cada coste de recepción tiene un receipt/order único; - un posting no se duplica por reintento; - moneda de precio, ledger y caja coincide; - suma de líneas coincide con total de quote; - un día cerrado no cambia por un segundo cierre; - save/load conserva caja y ledger; - rollback no deja stock vendido sin ingreso o ingreso sin venta; - los acumulados coinciden con postings o justifican su separación. # 20. Momentos de reconocimiento En el flujo Phase 1, el pedido se crea sin pagar y el coste se reconoce al recibir. En el modelo técnico, **SupplierReceivingEconomyService** también registra coste por cantidad recibida. El ingreso se reconoce al checkout completado. Esta consistencia

es valiosa: evita registrar mercancía que nunca llegó y venta que no terminó. Para balance futuro debe decidirse si el coste de recepción es flujo de caja, coste de inventario o ambos. El v0.2 proponía coste vendido; el runtime actual carga la compra completa al día de recepción. La diferencia afecta resultados diarios, rotación y punto de equilibrio. Sprint 17 debe elegir la métrica de UI sin alterar necesariamente el ledger de caja. # 21. Capital inicial histórico | Dificultad histórica | Capital | | — | — | | Relajada | 28.000 € | | Estándar | 20.000 € | | Exigente | 16.000 € | | Runtime actual | 1.000 € |

Los valores 28k/20k/16k pertenecen a la visión v0.1 con inversión inicial, alquileres y expansiones. El valor de 1.000 € procede de `CC_Sprint15Settings.asset` y alimenta el flujo actual. No debe interpretarse como corrección automática del diseño antiguo: el alcance y precios actuales son mucho menores. El tuning debe medir cuántos pedidos y errores permite la caja, no comparar cifras nominales sin contexto. # 22. Costes fijos e impuestos históricos | Concepto | Valor v0.1 | Estado actual | | — | — | — | | Alquiler | 150 €/día | no implementado | | Servicios | 25 €/día | no implementado | | Seguro/licencias | 15 €/día | no implementado | | Electricidad | 10–25 €/día | no implementado | | Impuesto | 10 % beneficio positivo / 7 días | no implementado | | Transporte normal | 50–100 €/pedido | no implementado | | Entrega urgente | 200 € | no implementado |

Estos números se conservan como visión. Introducirlos antes de medir el flujo de siete días podría convertir una economía representativa en insolvente. Su futura incorporación requiere ledger de tipos adicionales, UI, save, calendario, QA y parámetros de dificultad. # 23. Márgenes históricos y punto de equilibrio V0.1 fijó margen bruto medio objetivo de 35 %, punto de equilibrio aproximado de 600 €/día y rango saludable de 800–1.200 €/día con costes fijos cercanos a 210 €. Esos valores pertenecen a una tienda y duración de día distintas. El catálogo Phase 1 presenta márgenes brutos por producto de aproximadamente 28–55 %, pero el throughput y la caja inicial no permiten trasladar directamente el objetivo diario histórico. La referencia del 35 % sigue siendo útil como centro conceptual, no como obligación para cada SKU. Hardware puede aportar margen bajo y ticket alto; accesorios pueden financiar la operación. El balance debe evaluar cesta y frecuencia, no solo porcentaje individual. # 24. Dificultad histórica | Parámetro | Relajada | Estándar | Exigente | | — | — | — | — | | Capital | 28.000 | 20.000 | 16.000 | | Coste mercancía | ×0,95 | ×1,00 | ×1,05 | | Impuestos | 5 % | 10 % | 15 % | | Sensibilidad precio | ×0,85 | ×1,00 | ×1,15 | | Investigación | ×0,85 | ×1,00 | ×1,15 | | Pérdida reputación | ×0,75 | ×1,00 | ×1,25 |

No existe selector económico equivalente en el runtime actual. La dificultad no debe reintroducirse hasta que Estándar sea estable. Los requisitos estructurales no deberían variar; se ajustan colchón y presión, no se ocultan reglas ni se invalida la accesibilidad. # 25. Catálogo runtime Phase 1 | ID | Nombre | Coste unit. | Venta | Margen | Margen bruto | Uds/caja | Coste caja | | — | — | — | — | — | — | | game-neon-drift | Neon Drift | 15.00 € | 15.00 € | 0.00 € | 0.0 % | 12 | 180.00 € | | case-cloud-runner | Cloud Runner Case | 8.00 € | 8.00

€ | 0.00 € | 0.0 % | 16 | 128.00 € | | console-vertex-one | Vertex One Console
 | 180.00 € | 180.00 € | 0.00 € | 0.0 % | 2 | 360.00 € | | controller-orbit-pad
 | Orbit Pad Controller | 30.00 € | 30.00 € | 0.00 € | 0.0 % | 6 | 180.00 € | |
 headset-signal-pro | Signal Pro Headset | 45.00 € | 45.00 € | 0.00 € | 0.0 % | 4 |
 180.00 € | | accessory-memory-core | Memory Core Accessory | 9.00 € | 9.00 € |
 0.00 € | 0.0 % | 10 | 90.00 € |

Estos valores proceden de `ContentCatalog.asset`, referenciado por `RuntimeAssetRegistry.asset`. Son la autoridad de datos del flujo Phase 1 observado. Todos garantizan `sale > wholesale`. No existe precio editable ni proveedor alternativo dentro de esta capa. # 26. Catálogo técnico S13 | ID | Asset | Categoría | Tags | Venta técnica | | — | — | — | — | — | | collectible-cloud-hero | CloudHero | collectible | collectible, figure | 19.99 € | | cartridge-cloud-racers | CloudRacers | video-game | cartridge, retro, racing | 27.99 € | | memory-card-64 | MemoryCard64 | accessory | hardware, storage | 24.99 € | | console-nebula-8 | Nebula8 | console | hardware, retro, console | 89.99 € | | controller-orbit-pad | OrbitPad | accessory | hardware, controller | 34.99 € | | cartridge-pixel-quest | PixelQuest | video-game | cartridge, retro, adventure | 29.99 € |

Este catálogo alimenta el dominio técnico de productos, suppliers, prices y tests. Comparte `controller-orbit-pad` con Phase 1, pero le asigna otro precio; el resto de IDs difiere. No debe mezclarse en un mismo save o quote sin una tabla de migración/proyección. # 27. Proveedores técnicos y costes ## NebulaDistribution | Producto | Coste unit. | Uds/caja | Cajas | Coste caja | Venta | Margen unit. | | — | — | — | — | — | — | | cartridge-cloud-racers | 12.99 € | 6 | 1-12 | 77.94 € | 27.99 € | 15.00 € | | cartridge-pixel-quest | 11.99 € | 6 | 1-12 | 71.94 € | 29.99 € | 18.00 € | | console-nebula-8 | 79.99 € | 2 | 1-5 | 159.98 € | 89.99 € | 10.00 € | | controller-orbit-pad | 23.99 € | 4 | 1-8 | 95.96 € | 34.99 € | 11.00 € | | memory-card-64 | 8.99 € | 8 | 1-10 | 71.92 € | 24.99 € | 16.00 € |

PixelParcel

Producto	Coste unit.	Uds/caja	Cajas	Coste caja	Venta	Margen unit.
cartridge-cloud-racers	13.49 €	4	1-10	53.96 €	27.99 €	14.50 €
cartridge-pixel-quest	10.99 €	4	1-10	43.96 €	29.99 €	19.00 €
collectible-cloud-hero	15.99 €	3	1-8	47.97 €	19.99 €	4.00 €

Los catálogos prueban selección de proveedor, límites por cajas y costes autorita-

tivos. No contienen plazo, transporte, fiabilidad, stock de proveedor o descuentos por volumen. Esos conceptos históricos siguen diferidos. # 28. Mobiliario Phase 1 | ID | Nombre | Huella | Capacidad | Coste | Comprable | Productos | | — | — | — | — | — | — | | checkout-counter | Checkout Counter | 4×2 | 0 | 450.00 € | Sí | No | | wall-shelf | Wall Shelf | 4×1 | 24 | 180.00 € | Sí | Sí | | central-shelf | Central Shelf | 4×2 | 32 | 260.00 € | Sí | Sí | | low-display | Low Display | 3×2 | 12 | 160.00 € | Sí | Sí | | featured-display | Featured Display | 2×2 | 8 | 240.00 € | Sí | Sí | | backroom-storage | Backroom Storage | 5×2 | 80 | 220.00 € | No | No | | receiving-crate | Receiving Crate | 2×2 | 24 | 35.00 € | No | No | | decoration-plant | Decorative Plant | 1×1 | 0 | 25.00 € | No | No |

El coste de muebles forma parte del mismo flujo de pedidos y se paga al recibir. Muebles no comprables son infraestructura inicial o utilería. El precio del checkout, shelves y displays debe evaluarse contra caja inicial: con 1.000 €, una combinación básica consume una parte significativa del colchón. # 29. Capacidades técnicas de displays | Display | Capacidad lógica | Límite visible | Categorías | Placement | | — | — | — | — | — | | display-countertop-rack | 8 | 4 | accessory, collectible | technical-countertop-rack-2x2 | | display-floor-stand | 24 | 12 | console, video-game | technical-floor-stand-4x4 | | display-single-shelf | 12 | 6 | video-game, accessory, collectible | technical-shelf-4x2 |

Estas definiciones S8 no coinciden uno a uno con las capacidades del **ContentCatalog** Phase 1. La futura unificación debe decidir si el límite visual es independiente de la capacidad vendible y cómo se proyecta a modelos representativos. El stock lógico siempre es la fuente de verdad. # 30. Perfiles técnicos de clientes | Perfil | Preferencias | Peso | Paciencia | Paradas | Velocidad | | — | — | — | — | — | — | | customer-casual-browser | video-game, accessory | 5 | 45 s | 3 | 1.8 | | customer-collector | collectible, card-code | 2 | 60 s | 4 | 1.5 | | customer-focused-buyer | console, video-game | 4 | 30 s | 2 | 2.1 | | customer-impatient-visitor | accessory, video-game | 3 | 15 s | 1 | 2.4 |

El spawn técnico admite hasta 6 clientes, con intervalo de 8 segundos. Estos perfiles expresan variedad de navegación y paciencia, pero no contienen presupuesto, probabilidad de compra ni sensibilidad al precio. La Phase 1 simplificada completa una venta determinista desde el primer display con stock. # 31. Autoridad de valores y plan de unificación Para H6 debe existir un conjunto de valores candidato. La recomendación es que el Content Catalog o un nuevo catálogo consolidado sea autoridad runtime y que los modelos S6–S13 consuman/proyecten esos mismos IDs. Alternativamente, Phase 1 puede retirarse tras migrar a los servicios de dominio. En ambos casos se requiere migración de saves, actualización de tests, catálogos y documentación. 1. inventariar IDs duplicados y equivalencias; 2. elegir catálogo canónico; 3. migrar precios/costes y referencias; 4. crear validación de cobertura supplier/price/product; 5. actualizar RuntimeAssetRegistry; 6. probar save viejo y nuevo; 7. eliminar o marcar assets obsoletos; 8. registrar ADR y evidencia. # 32. Precios de venta El catálogo Phase 1 incluye precio de venta junto al producto. El modelo S13 separa **ProductSalePriceCatalog** de **ProductDefinition**, decisión más flexible para promociones y mercados. La se-

paración debe conservarse en la arquitectura final: identidad/categoría no debería cambiar al ajustar precio. Phase 1 puede proyectar sus valores a un catálogo separado durante la unificación. Todos los precios deben ser positivos y tener moneda. Ausencia de precio bloquea quote antes de mutar stock. No se permite fallback silencioso a coste, cero o precio recomendado. # 33. Costes de compra En Phase 1 el coste wholesale está en el producto y no depende de proveedor. En el modelo técnico el coste pertenece a `SupplierCatalogEntry`, permitiendo alternativas. La segunda opción es la dirección preferida para crecimiento. El coste registrado en un pedido debe congelarse en la línea para que cambios posteriores del catálogo no reescriban pedidos existentes. Los costes deben incluir claramente si representan unidad, caja, transporte o total. El código Phase 1 usa coste de caja para pedidos de producto (`wholesale × unitsPerCase`) y coste unitario para muebles; `Phase1OrderRecord.UnitCostCents` significa coste por unidad pedida, cuya semántica depende de `IsFurniture`. Esta ambigüedad merece nombres separados. # 34. Precio editable y promociones La visión v0.1 permitía precio entre 50 % y 150 % del recomendado y promociones de 5–20 % durante 1–7 días. Ninguna de estas reglas está implementada. No deben aparecer como disponibles en H6. Su futuro diseño requiere precio base, precio del jugador, validez temporal, sensibilidad, redondeo, UI, save y telemetría. Antes de activar precios editables, el sistema debe demostrar una demanda comprensible. Un slider sin feedback convierte el tuning en adivinación. El jugador necesita precio de coste, margen, referencia, demanda estimada y consecuencia. # 35. Capital inmovilizado y valor de inventario La mercancía recibida reduce caja inmediatamente, pero no existe un cálculo runtime de valor de inventario. Para análisis se define `valor_coste_stock = Σ unidades_on_hand × coste_unitario_congelado`. Si el coste varía por proveedor, se necesita política de valoración —promedio ponderado, FIFO u otra simplificación— solo si afecta decisiones. Para H6 puede bastar con caja, gasto acumulado y unidades. El inventario inmovilizado es stock que no rota dentro de la ventana. La métrica debe separar warehouse y display, y no penalizar mercancía recién recibida. Su objetivo es detectar sobrestock, no imponer contabilidad realista innecesaria. # 36. Flujo de caja y solvencia El runtime impide recibir un pedido si la caja es insuficiente. No reserva fondos al pedir, por lo que múltiples pedidos pendientes pueden superar el saldo. Esta regla puede crear una decisión útil o una trampa confusa. La UI debe mostrar coste comprometido total y “caja tras recepciones”. Se define solvencia operativa como capacidad de cubrir recepciones necesarias y continuar vendiendo. El balance no debe permitir que una única compra razonable deje al jugador sin ningún SKU vendible ni vía de recuperación, salvo decisión deliberada claramente advertida. # 37. Liquidez mínima y protección contra softlock La campaña candidata debe conservar rutas de recuperación: vender stock existente, recibir parcialmente si la regla lo permite, cancelar pedidos no pagados, retirar muebles y recuperar stock, o reiniciar desde save. No se introducirá préstamo automático sin diseño. Una reserva de caja recomendada puede mostrarse, pero no debe bloquear arbitrariamente. Un softlock económico ocurre cuando no hay producto vendible, caja para recibir, pedido cancelable ni acción productiva. QA debe construir ese estado y confir-

mar prevención o recuperación documentada. # 38. Coste de oportunidad El coste de oportunidad representa ventas perdidas por display vacío, stock en warehouse sin reponer, cola saturada o capital atrapado. No necesita convertirse en dinero del ledger; puede ser métrica de análisis. Se calcula a partir de demanda no satisfecha y margen potencial, siempre marcado como estimación. La UI de lanzamiento puede mostrar motivos simples —sin stock, cola llena, precio rechazado—. El detalle cuantitativo puede reservarse a informes posteriores, evitando sobrecargar el vertical slice. # 39. Margen bruto y markup La especificación usa dos medidas distintas: $\text{margen_bruto_pct} = (\text{venta} - \text{coste}) / \text{venta}$ y $\text{markup_pct} = (\text{venta} - \text{coste}) / \text{coste}$. Deben nombrarse correctamente. El catálogo Phase 1 oscila aproximadamente entre 28 % y 55 % de margen bruto. El técnico incluye extremos de 11 % para consola y 64 % para memoria, lo que puede ser intencional como combinación de tráfico y rentabilidad. No se equilibra exigiendo el mismo porcentaje. Se evalúa contribución por espacio, capital, velocidad de venta, riesgo de stockout y complementariedad. # 40. Resultado diario y beneficio `DailyResultsService` solo crea resultado cuando el día está cerrado y exige que el número de postings de checkout coincida con checkouts completados. Suma ingreso y coste de recepción del día y obtiene resultado bruto. Es una comprobación fuerte de coherencia, pero no un P&L completo. Para Sprint 17 se decidirá qué panel muestra: flujo de caja diario, margen de ventas, coste de mercancía vendida, gastos fijos y beneficio. Puede haber varias métricas, siempre con etiquetas precisas. No se llamará “beneficio” al flujo de caja si una recepción de inventario distorsiona el periodo. # 41. Modelo de inventario El inventario utiliza contenedores con capacidad por unidades, stacks por producto y `Quantity` no negativa. Las operaciones devuelven resultados y razones, no dependen de excepciones para fallos esperables. Warehouse, displays y reservas representan estados distintos; una unidad no debe contabilizarse simultáneamente como disponible y comprometida. La capacidad es una simplificación deliberada: una unidad consume una unidad salvo que se amplíe el modelo. El concepto histórico `displaySize` no está implementado. Introducir volumen por producto requeriría migración y revalidación de todos los contenedores. # 42. Conservación de unidades `InventoryTransferService` captura cantidades antes, valida origen, destino, definición, disponibilidad y capacidad, muta ambos contenedores y comprueba que la suma de capacidades usadas no cambió. Esta es la invariante central: una transferencia solo cambia ubicación. Si cualquier precondition falla, ambos estados permanecen iguales.

```
unidades_origen_antes + unidades_destino_antes
= unidades_origen_después + unidades_destino_después
```

43. Capacidad y stacks

El ADR-0024 fija capacidad en unidades y stacks por producto. Una adición puede crear o ampliar stack hasta capacidad; una retirada no puede exceder on-hand. La capacidad disponible es $\text{capacity} - \text{used}$. No se acepta overflow,

cantidad cero como mutación válida ni producto inexistente cuando el registro lo exige. Los valores de warehouse integrado usan capacidad 200 en el mutador Phase 1; las definiciones de muebles y displays tienen capacidades menores. Ese 200 es un parámetro técnico y debe trasladarse a settings o catálogo si se mantiene. # 44. Reservas y disponibilidad La disponibilidad se calcula como **on_hand - reserved**. Una reserva identifica cliente, carrito, display, producto, cantidad y estado. Solo reservas activas se añaden al carrito. El checkout consume reservas; abandono o cierre las libera. La procedencia evita que una reserva se cobre desde otro display o cliente. La configuración técnica limita a una unidad por reserva y tres por carrito. Son valores representativos. El balance futuro puede permitir cantidades mayores, pero antes debe validar UI, capacidad, stock y comportamiento de búsqueda. # 45. Stock de warehouse, display y compromiso | Estado | Disponible para | Regla | | — | — | — | | Warehouse | reposición | no vendible directamente | | Display on-hand | búsqueda/reserva | limitado por capacidad | | Reservado | cliente propietario | no disponible para otros | | Carrito | checkout | respaldado por reserva | | Vendido | ninguno | retirado tras commit | | Pedido | futuro | no es stock hasta recepción |

La UI no debe sumar todos los estados como “stock disponible”. Debe mostrar total físico, disponible y reservado cuando sea relevante. El cierre diario no puede eliminar unidades; solo libera compromisos no cobrados. # 46. Pedidos por cajas El modelo técnico crea órdenes con líneas únicas por producto y cantidad entera de cajas. Cada supplier entry define unidades por caja y mínimo/máximo. Se rechazan request vacío, producto duplicado, producto no ofrecido y cajas fuera de límites. El coste unitario queda copiado en la línea. Phase 1 usa **caseQuantity** para productos y cantidad de muebles para furniture. La interfaz debe distinguir ambos. “2” no puede significar dos unidades en un panel y dos cajas en otro sin etiqueta. # 47. Recepción atómica La recepción valida estado y capacidad antes de confirmar. El ADR-0029 exige caja completa como unidad de recepción técnica. En Phase 1, recibir todo el pedido comprueba caja, añade stock, registra gasto, actualiza save y emite feedback. Si no hay caja, el pedido permanece ordenado. La transacción debe mantener coherencia entre cash, stock, order state y ledger. Cualquier fallo durante commit requiere rollback o escritura indivisible. Las pruebas deben inyectar fallo, no solo comprobar happy path. # 48. Momento de pago de pedidos El runtime paga en recepción. Esta regla evita cobrar pedidos que no llegan y simplifica lead time, pero permite compromisos no financiados. El documento la marca como **RUNTIME / CANDIDATA A REVISIÓN**, no como decisión definitiva. Alternativas: pagar al pedir; reservar fondos; depósito y saldo; crédito de proveedor. Cada opción altera UX y riesgo. Para H6 puede mantenerse pago en recepción si la UI muestra coste total y la campaña no permite acumulación confusa. No se introducirá crédito antes de H6. # 49. Proveedores El modelo técnico incluye Nebula Distribution y Pixel Parcel con catálogos parcialmente solapados. La selección permite comparar coste y tamaño de caja. No hay reputación, lead time, fiabilidad ni descuentos. Phase 1 omite identidad del proveedor y usa coste propio del producto. Sprint 17 debe decidir si la candidata requiere elección real de proveedor o solo un flujo

único. Una comparación sin diferencias significativas añade UI y QA sin decisión. Si se conserva, al menos un producto compartido debe generar un trade-off comprensible —coste frente a caja, disponibilidad o entrega—. # 50. Displays y asignación de producto Cada display técnico admite una única asignación de producto. La capacidad y categorías permitidas se validan. Reponer transfiere unidades desde warehouse al contenedor del display de forma atómica. Phase 1 mantiene producto y cantidad en el fixture y sincroniza inventario integrado. La asignación única mejora legibilidad y evita mezclas difíciles de representar. Un cambio de producto requiere que el estado anterior no deje stock huérfano. La retirada de mueble devuelve producto al warehouse en Phase 1. # 51. Representación visible de stock El límite visual puede ser menor que la capacidad lógica. La Art Bible define densidad visual; la economía conserva cantidad real. Nunca se deducirá stock contando meshes. Los umbrales históricos de vacío/casi vacío/medio/lleno son útiles como representación, no como lógica. La sincronización debe actualizar después de commit y tolerar pooling o LOD. Un fallo visual no puede mutar el inventario; un fallo de inventario debe mostrar estado coherente y bloquear venta. # 52. Reposición La reposición manual debe verificar asignación, producto, stock warehouse, capacidad destino y cantidad. El resultado es todo o nada. El feedback indica unidades movidas o razón de rechazo. La frecuencia de reposición forma parte del balance espacial: displays pequeños aumentan trabajo y stockouts; grandes inmovilizan capital y espacio. El tuning de Sprint 17 medirá tiempo entre reposiciones, ventas perdidas y porcentaje de capacidad utilizado. No se optimizará solo para eliminar trabajo; la reposición es parte de la fantasía operativa. # 53. Demanda actual El flujo Phase 1 no implementa una curva probabilística de demanda: completa una compra determinista del primer display con stock. El modelo técnico de shopping busca de forma determinista según intent y categorías. Los perfiles aportan preferencia, peso y paciencia, pero no presupuesto o elasticidad. Por tanto, los precios actuales no están balanceados contra sensibilidad. Son valores de transacción. Sprint 17 necesita una campaña controlada antes de activar cualquier fórmula de demanda más rica. # 54. Demanda futura y legibilidad Una demanda candidata puede combinar preferencia, disponibilidad, precio relativo, presupuesto, popularidad y paciencia, pero debe producir motivos observables. La fórmula histórica de interés no debe copiarse ciegamente. Se empieza con pocos factores, se registra resultado y se comprueba que el jugador puede aprender.

score = preferencia + disponibilidad + atractivo - penalización_precio - fricción compra si score >= umbral y presupuesto >= total

La aleatoriedad se limita a selección dentro de rangos; el mismo seed y estado debe ser reproducible en tests. # 55. Perfiles de cliente Los cuatro perfiles técnicos —Casual Browser, Collector, Focused Buyer e Impatient Visitor— varían preferencias, peso, paciencia, paradas y velocidad. El reparto de pesos 5/2/4/3 implica mezcla relativa 35,7/14,3/28,6/21,4 % antes de filtros. Estos porcentajes son inferencia de pesos, no evidencia de spawn observado. El balance debe comprobar que ningún perfil domina por rendimiento, navegación o disponibilidad de categorías. Collector prefiere categorías que no están completas en

todos los catálogos; esa cobertura debe validarse. # 56. Spawn y capacidad de tienda El asset técnico admite 6 clientes activos y llegada cada 8 segundos. **StoreRuntimeSettings** mantiene otro límite representativo de blockout, por lo que el número efectivo depende de la composición. El balance debe usar el límite real de la build candidata, no un asset aislado. La tasa de llegada se compara con throughput de compra y checkout. Si la capacidad es seis y la cola también seis, la tienda puede saturarse de forma distinta según duración de browsing y paciencia. El tuning debe medir ocupación, no solo spawns. # 57. Paciencia La paciencia técnica oscila entre 15 y 60 segundos. El servicio reduce tiempo y desencadena frustración/abandono según estado. El valor debe evaluarse frente a navegación, browsing, cola y duración del día. Una paciencia de 15 segundos puede ser suficiente en escenario técnico y agresiva en escena representativa. La paciencia no debe ocultar fallos de navegación ni compensarse con valores enormes. Primero se corrige camino, click-through y checkout; después se ajusta. # 58. Presupuesto y sensibilidad al precio No están implementados en los perfiles actuales. El v0.1 los contemplaba como atributos. Su incorporación es post-H6 salvo decisión formal. Deben expresarse en rangos y moneda, evitar discriminaciones no intencionadas y generar feedback de “demasiado caro” o “sin presupuesto” distinto de “sin stock”. El presupuesto no debe convertir todos los productos caros en ventas imposibles. Se diseña junto con mix de perfiles, cesta y disponibilidad. # 59. Intención y búsqueda La shopping session separa intención, carrito y estado. La búsqueda determinista prioriza categorías y permite fallback según settings. Esto hace reproducibles los tests. El balance controla cuántas paradas, categorías y unidades busca cada perfil; la arquitectura garantiza que la selección no depende de orden accidental de colecciones. La falta de producto debe producir abandono o alternativa de forma explícita y medible. No se generará una venta de fallback sin informar la lógica. # 60. Carrito El carrito técnico tiene capacidad 3 unidades y reserva máxima 1 por operación. Rechaza reserva inactiva, propiedad incorrecta, duplicada o exceso de capacidad. La cesta Phase 1 simplificada vende una unidad. La capacidad de tres es representativa y debe revisarse con duración, precio y stock. El carrito es una promesa respaldada por reservas; no es una lista visual desconectada. Al retirar línea se libera o gestiona reserva según servicio. La UI muestra unidades, no solo líneas. # 61. Abandono y liberación Un cliente puede abandonar por paciencia, falta de producto, cierre o cancelación. Todas las reservas activas deben liberarse, el carrito quedar consistente y la cola eliminar entrada si existe. No se registra ingreso. La métrica distingue abandono por causa para no atribuir a precio un fallo de navegación. El cierre determinista drena clientes y resuelve compromisos. Cualquier reserva persistente después de Closed es defecto bloqueante de integridad. # 62. Cola FIFO La cola técnica tiene capacidad 6 y orden FIFO estricto. La estación procesa una entrada. Se rechazan duplicados, cola llena, estado inválido y mismatch. El balance evalúa espera y abandono, pero no puede saltarse FIFO para mejorar cifras. El tamaño seis no es objetivo final. Debe encajar físicamente en StoreInitial y en la UI. Una cola invisible o que atraviesa mobiliario invalida el tuning. # 63. Estación de checkout Existe una estación técnica de entrada única, abierta al inicializar. La estación y

cola deben concordar sobre entry actual. El checkout solo comienza en estado Processing. El throughput futuro puede ajustarse mediante tiempo de servicio o empleados, pero el vertical slice prioriza corrección. No se deben añadir múltiples cajas antes de medir la estación única. La visión futura puede ampliar capacidad, pero requiere routing, cola y balance nuevos. # 64. Quote económico **CheckoutQuoteService** recorre líneas del carrito, busca precio por producto y crea total en la moneda del catálogo. Un carrito vacío o precio ausente falla antes de mutar stock. Cada línea conserva cantidad, precio unitario y subtotal. El quote es la autoridad monetaria de la transacción. La UI puede previsualizar el quote, pero el commit debe usar la misma instancia o recomputar bajo versión controlada. Un cambio de precio entre preview y pago requiere política explícita. # 65. Preflight de checkout **CheckoutService** valida transaction ID, cart no completado, cola, estación, sesión, propiedad, reservas, displays, producto y stock agregado por display. Solo después registra transacción y comienza mutación. El objetivo es impedir ventas parciales y errores tardíos. La preflight debe incluir cualquier nueva condición económica antes de commit: moneda, price version, impuestos, descuento o fondos de cliente. No se añade una validación después de retirar stock si puede hacerse antes. # 66. Commit y rollback de checkout El servicio retira stock, quita líneas, consume reservas, marca sesión, completa cola, estación y transacción. Implementa rollback para stock, carrito y reservas en varios fallos. Algunos fallos posteriores de cola/estación/transacción no muestran rollback completo en el fragmento observado; las pruebas existentes deben cubrir la imposibilidad de llegar a esos estados o justificar el contrato. La atomicidad interagregado debe validarse con inyección de fallos. No basta con asumir que preflight hace imposible todo fallo. Si el ledger falla después de checkout, la capa económica debe evitar estado vendido sin ingreso o implementar compensación. # 67. Idempotencia de checkout La idempotencia usa transaction ID, completed carts y posting key. Repetir la misma operación no debe retirar otra unidad ni registrar otro ingreso. ADR-0073 añade snapshots de checkouts completados a persistencia para conservar la protección tras save/load. Los IDs deben ser estables y únicos por venta. El escenario Phase 1 deriva ID de día y secuencia de cliente. Una restauración no puede reiniciar secuencia y colisionar. # 68. Ledger **EconomyLedger** almacena entries positivas con **EconomyPostingKey** de tipo y source ID. Rechaza entry ID duplicado, posting duplicado y moneda incorrecta. Ordena por ID y permite totales y conteos por día/tipo. Los importes son magnitudes positivas; el tipo determina ingreso o coste. Esta elección simplifica el vertical slice, pero futuras categorías necesitarán dirección o tipo. No se representará gasto con importe negativo si el constructor exige positivo; se añadirá posting type o clasificación explícita. # 69. Posting de ingresos Una venta económica crea posting **CheckoutRevenue** con source transaction ID y total del quote. Antes comprueba que el posting no existe. El ledger y price catalog deben compartir moneda. El recuento diario se compara con checkouts completados. La Phase 1 integrada añade además cash y records persistidos. La unificación debe evitar que ambas rutas registren dos veces el mismo checkout si coexisten. # 70. Posting de costes La recepción crea posting **SupplierReceivingCost** con receipt ID y coste unitario por cantidad

recibida. Reintentos se rechazan. Phase 1 usa order ID como source y coste total. Esta consistencia permite deduplicación. El coste debe corresponder exactamente a unidades añadidas. Si una recepción futura es parcial, cada parcial necesita source único o cantidad acumulada controlada. # 71. Caja y ledger La caja y el ledger son dos representaciones relacionadas: saldo mutable y historial. La operación debe actualizarlas juntas. El save integrado persiste ambas. QA debe reconstruir saldo esperado desde saldo inicial y postings o, si existen categorías no registradas, documentar la diferencia. Los acumulados Phase 1 **LifetimeRevenueCents** y **LifetimeExpenseCents** son una tercera vista. Mantener tres fuentes sin reconciliación aumenta riesgo; Sprint 17 debe definir cuál es derivada y cuál autoritativa. # 72. Ciclo diario El runtime de Sprint 15 configura día de 300 segundos; el asset técnico de DayCycle usado para escenarios tiene 8 segundos. La visión histórica hablaba de una hora real a velocidad $\times 1$ y apertura 08:00–22:00. Son tres escalas distintas. La build candidata debe elegir una y actualizar UI, tests y documentación. El agregado StoreDay controla estados, tiempo y actividad. El balance usa duración para calcular llegadas, oportunidades de venta, reposición y presión de cola. Cambiarla obliga a retunar casi todo. # 73. Apertura y admisión La apertura permite clientes y operaciones. El cierre bloquea nuevas admisiones mediante gates. No debe entrar un cliente después de BeginClosing aunque un timer dispare. La UI muestra estado y aviso. La Phase 1 puede autoabrir; el diseño histórico preveía apertura manual. Sprint 17 debe fijar el flujo candidato. # 74. Drain y cierre Al cerrar, se procesan o cancelan clientes de forma determinista, se liberan reservas y se sella cola. El día solo pasa a Closed cuando el snapshot de readiness confirma que no quedan obligaciones. Esta regla protege save y resumen. No se debe forzar cierre borrando entidades. Un timeout puede activar compensaciones documentadas, pero debe conservar inventario y transacciones. # 75. Resumen diario El resumen técnico incluye ingresos de checkout, costes de recepción, resultado bruto, postings, clientes, checkouts y segundos abiertos. La visión histórica añadía gastos, salarios, impuestos, ventas por producto e incidencias. Para H6 se debe mostrar solo lo respaldado por datos, con posibilidad de ampliar después. La discrepancia entre checkouts y postings bloquea creación del resultado. Es un gate valioso y debe mantenerse en save/load. # 76. Autosave de día cerrado El autosave posterior al cierre es idempotente: un segundo intento no duplica snapshot ni cambia economía. Solo se guarda un estado coherente. Un fallo debe mostrar feedback y permitir reintento sin repetir ingresos o costes. La campaña de siete días debe cerrar, guardar, salir, cargar y continuar varias veces; no basta con mantener una sesión en memoria. # 77. Persistencia económica **IntegratedGameStateSnapshot** schema 2 conserva cash, inventarios, supplier orders, displays, clientes, shopping sessions, reservas, cola, checkout, transactions y ledger. La Phase 1 agrega su estado/proyección. La escritura validada, checksum, generation, backup-first y restore en dos fases protegen integridad. Los cambios de catálogo no pueden invalidar silenciosamente IDs guardados. Una migración debe resolver productos/muebles retirados, precios históricos y orders pendientes. # 78. Compatibilidad y migraciones Una save conserva el precio/coste de la transacción o pedido cuando sea necesario. Si solo guarda

ID y consulta catálogo nuevo, un balance patch puede reescribir historia. Las órdenes ya creadas guardan coste unitario, lo cual es correcto. Las transacciones persistidas deben guardar total/lineas si la auditoría lo requiere. La unificación de los dos catálogos necesita mapa de IDs y política para saves Phase 1. No se renombrarán IDs sin migración y tests. # 79. Orden de operaciones económico

1. validar entrada y estado
2. resolver datos y precios
3. construir quote/plan
4. validar capacidad, stock, caja e idempotencia
5. preparar cambios
6. commit lógico y físico
7. registrar ledger/caja
8. persistir snapshot
9. emitir feedback
10. archivar evidencia

El feedback no antecede al commit. El sonido de ingreso, VFX y toast solo se emiten tras éxito. Si persistencia es asíncrona en el futuro, se distingue transacción confirmada de autosave pendiente. # 80. Errores y recuperación Cada fallo debe conservar estado y dar razón: cash insuficiente, stock insuficiente, capacity, missing price, duplicate posting, invalid state, queue full o currency mismatch. El usuario recibe una acción recuperable; los detalles técnicos van a log. No se convierten excepciones de programación en mensajes genéricos sin evidencia. Los fallos repetibles deben tener tests de antes/después. Una operación fallida no aumenta generation ni altera save salvo registro de diagnóstico no económico. # 81. Parámetros de balance | Familia | Parámetros | | — | — | | Caja | inicial, reserva recomendada, compromisos | | Productos | coste, venta, unidades/caja, categoría | | Muebles | coste, capacidad, huella | | Suppliers | catálogo, cajas, límites, entrega futura | | Clientes | peso, paciencia, paradas, velocidad, presupuesto futuro | | Flujo | llegadas, activos, cola, día | | Economía | gastos futuros, impuestos, dificultad | | QA | seed, duración, thresholds, métricas |

Cada parámetro debe residir en un asset o configuración identificable. Los defaults de código sirven para authoring seguro, no para ocultar valores runtime. # 82. Valores hardcodeados, serializados y derivados La auditoría distingue: serializados en assets (**initialCash**, precios, costes, capacidades); hardcoded en servicios o mutadores (warehouse capacity 200, IDs de contenedor, nombres de posting); derivados (**margin**, total, available); y temporales de tests. Sprint 17 prioriza mover hardcodes de balance a configuración sin convertir IDs técnicos estables en sliders. Un valor se considera tunable si cambiarlo no exige modificar una regla. La razón de cada hardcode debe documentarse. # 83. Metodología de tuning 1. congelar build, seed y catálogo; 2. definir hipótesis y métrica; 3. cambiar una familia de parámetros; 4. ejecutar varias campañas; 5. registrar distribución, no solo promedio; 6. revisar experiencia cualitativa; 7. comparar contra baseline; 8. aceptar, rechazar o iterar; 9. actualizar trazabilidad; 10. repetir Golden Path. No se ajustan números durante una campaña y luego se

combinan resultados. No se optimiza exclusivamente para que el autor experto gane. Se incluyen rutas prudente, agresiva y de error razonable. # 84. Métricas mínimas - caja inicial/final y mínima; - ingresos y costes por día; - ventas y margen por producto; - unidades recibidas, warehouse, display, reservadas y vendidas; - stockouts y duración; - pedidos pendientes y coste comprometido; - clientes llegados, compraron, abandonaron; - tiempo de cola y ocupación; - reposiciones y tiempo operativo; - save/load y discrepancias; - acciones fallidas por razón; - resultado acumulado a siete días. # 85. Rotación de stock **rotación = unidades vendidas / stock medio**, calculada por producto y ventana. El stock medio puede aproximarse con snapshots de inicio/fin para H6, siempre documentado. La rotación alta con stockouts no es necesariamente buena; la baja puede ser aceptable en ticket alto. Se analiza junto con margen y capital. # 86. Stockout Un stockout se registra cuando existe intención/demanda y disponibilidad cero, no simplemente display vacío sin clientes. Se mide frecuencia, duración y ventas perdidas estimadas. El objetivo no es cero absoluto: cierto riesgo hace relevantes pedidos y reposición. Debe ser recuperable y legible. # 87. Sobrestock Sobrestock combina unidades inmóviles, días de cobertura y porcentaje de caja invertida. La caja pequeña hace especialmente peligroso pedir cajas grandes. La elección de supplier o tamaño de caja puede modular el riesgo. La campaña debe permitir corregir exceso mediante ventas, no exigir venta de sobrantes antes de H6. # 88. Throughput y cuellos de botella El throughput de ventas está limitado por llegada, búsqueda, stock, cola y checkout. Si una etapa es mucho más lenta, cambiar precios no resuelve el problema. El profiling económico registra tiempos y estados. StoreInitial puede alterar distancias y, por tanto, resultados; el balance final se ejecuta en la escena candidata. # 89. Mix de productos La cesta de seis productos debe ofrecer diferencias de ticket, margen, caja y capacidad. Un producto dominante se detecta si maximiza beneficio, rotación y riesgo bajo simultáneamente. La consola puede actuar como ticket alto y margen bajo; accesorios como margen alto. El catálogo debe evitar que un único SKU cubra todas las estrategias. # 90. Elección de proveedor En la capa técnica, Pixel Parcel ofrece Pixel Quest más barato y cajas más pequeñas, mientras Nebula ofrece otros productos. Cloud Racers tiene coste ligeramente inferior en Nebula pero caja mayor. Este es un trade-off útil: precio frente a capital por caja. Debe preservarse o eliminarse conscientemente al unificar. # 91. Tamaño de caja El coste total de caja determina barrera de entrada. En Phase 1, cajas van de 90 € a 360 €. Con caja inicial de 1.000 €, varias compras son posibles pero una consola consume 36 %. El tuning debe medir cuánto margen queda para mobiliario y errores. No se cambia unidades/caja sin revisar warehouse, UI y save. # 92. Mobiliario y economía espacial Los muebles cuestan 160-450 € si son comprables y ocupan huellas distintas. Su retorno depende de capacidad y productos soportados. El checkout es necesario pero no genera capacidad; shelves convierten espacio en inventario visible. El balance debe evitar que el mueble más barato sea siempre superior por capacidad/coste o que uno caro no tenga ventaja. La comparación incluye legibilidad, circulación y línea de visión; no se reduce a ratio numérico. # 93. Duración del día y ritmo Con 300 segundos y llegada cada 8, el máximo teórico de arrivals antes de límites sería

37–38, pero browsing, capacidad y cierre reducen la cifra. Esta es una estimación, no resultado. Un día de cinco minutos puede ser apropiado para vertical slice y muy corto para lanzamiento. Sprint 17 debe evaluar fatiga y posibilidad de completar preparación/ventas. # 94. Campaña de siete días La campaña comprueba continuidad, no solo suma de siete escenarios. Debe comenzar desde nueva partida, ejecutar pedidos, recepción, placement, reposición, ventas, cierre, autosave y carga durante siete días, con al menos un error recuperable. Se registra caja, stock, orders, ventas y defectos cada día. Criterio inicial: no softlock, no pérdida/duplicación, todos los días cerrables, save consistente y más de una decisión viable. Los thresholds de rentabilidad se fijarán tras primera campaña, no se inventan de antemano. # 95. Golden Path económico 1. crear slot y confirmar caja inicial; 2. pedir mobiliario/producto; 3. recibir y verificar coste/caja; 4. colocar display; 5. asignar y reponer; 6. generar cliente/venta; 7. confirmar stock, cash, transaction y ledger; 8. cerrar día y revisar resumen; 9. guardar, volver al menú y cargar; 10. repetir y confirmar idempotencia. Cada paso captura estado anterior y posterior. Golden Path no sustituye escenarios negativos. # 96. Escenario nominal Un escenario nominal usa un display, dos productos, cash suficiente y una cola no saturada. Debe producir pedidos, recepciones, reposición y varias ventas sin fallos. Sirve para comparar cambios de precio o día. El seed, build y catálogo se registran. # 97. Escenario de caja baja Crear pedidos cuyo total supera caja, intentar recibir, confirmar rechazo sin stock/gasto, completar ventas existentes y volver a recibir si alcanza. La UI debe mostrar qué pedido es asequible. El estado no debe perder order ni duplicar gasto. # 98. Escenario de capacidad Llenar warehouse o display, intentar recibir/reponer, comprobar rechazo atómico, liberar capacidad y repetir. La suma de unidades permanece. El mensaje distingue capacidad de stock insuficiente. # 99. Escenario de precio ausente Eliminar una entrada de sale price en entorno de prueba, llevar el producto al carrito y ejecutar quote. Debe fallar antes de stock/queue/session commit, registrar missing product ID y permitir corregir catálogo. No se cobra cero ni usa otro precio. # 100. Escenario de duplicación Repetir transaction ID, cart completado, receipt ID y posting key después de save/load. Todos los reintentos deben ser no-op o fallo explícito. Cash, stock, ledger y counts permanecen. Este escenario cubre autosave, UI double-click y retries. # 101. Escenario de moneda incompatible Construir price catalog en otra moneda o ledger incompatible. Quote puede existir, pero checkout económico debe bloquear antes de mutación. La UI no ofrece conversión. La save no admite monedas mezcladas. El error se considera configuración, no decisión del jugador. # 102. Escenario de cierre con actividad Iniciar cierre con clientes, reservas y cola. Bloquear admisión, resolver o cancelar, liberar stock, completar/cancelar checkout según política, llegar a Closed y generar resultado. Ninguna reserva activa, entry processing o station busy puede persistir. # 103. Escenario save/load económico Guardar con order pendiente, stock warehouse/display, reserva, cash y ledger; cargar y comparar. Repetir con snapshot cerrado y backup recovery. Verificar sequence IDs para que nueva venta no colisione. Una diferencia de catálogo debe fallar con diagnóstico o migrarse, no ignorarse. # 104. Escenario de cambio de catálogo Duplicar build con un precio o coste modificado y cargar save anterior.

Orders existentes conservan coste congelado; nueva orden usa valor nuevo. Una transacción histórica conserva importe. Se comprueba que el balance patch no reescribe ledger. # 105. Pruebas automatizadas existentes En el subconjunto extraído se contabilizan 702 atributos de test en 69 archivos relacionados con economía y sistemas dependientes. No equivale al total oficial de la suite, que sigue siendo 1215 EditMode + 70 PlayMode en la baseline observada. El conteo por área se incluye abajo.

Área	Archivos	Atributos	Test/TestCase
—	—	—	—
Authoring	1	6	—
Checkout	10	105	—
Customers	12	96	—
DayCycle	12	143	—
Economy	10	100	—
Inventory	3	42	—
Orders	2	20	—
Other	1	1	—
Persistence	6	80	—
Shopping	9	79	—
Suppliers	2	18	—
UIUX	1	12	—

La cobertura estructural es fuerte en invariantes, pero no demuestra balance perceptual, estrategia, ritmo ni campaña externa. # 106. Pruebas manuales Las pruebas manuales deben observar comprensión y decisiones: si el jugador sabe por qué no puede recibir, si identifica producto rentable, si repone a tiempo, si entiende caja y si el resumen explica el resultado. Se registra acción, expectativa, resultado y comentario, no solo PASS global. # 107. Simulación y harness de balance Se recomienda un simulador determinista fuera de Presentation que consume catálogos, parámetros y políticas para ejecutar miles de días simplificados. No sustituye el juego: detecta extremos, insolvencia y dominancia. Debe usar las mismas fórmulas y exportar seed/config. Cualquier modelo aproximado se etiqueta. El harness no debe acceder a Unity GameObjects. Puede vivir en Domain/Application tests o herramienta separada. # 108. Muestreo y confianza Un único run no permite concluir. Para variables deterministas basta cubrir combinaciones; para aleatoriedad se ejecutan múltiples seeds y se reportan mediana, percentiles y fallos. El número de runs depende de variabilidad. Se evita falsa precisión y se conservan datos brutos. # 109. Criterios de aceptación de Sprint 16 - ContentCatalog y runtime integran costes/precios sin referencias rotas; - StoreInitial muestra UI económica legible; - pedir, recibir, colocar, reponer y vender funciona; - feedback de revenue/expense ocurre tras commit; - no hay pérdida/duplicación; - save conserva cash/stock/orders/ledger; - placeholders y dos capas de catálogo están documentados; - build post-integración y Golden Path pendientes hasta ejecución. Sprint 16 valida presentación representativa, no balance final. # 110. Objetivos de Sprint 17 1. unificar catálogos y autoridad; 2. fijar caja, día, precios, costes y capacidades candidatos; 3. ejecutar campaña de siete días; 4. medir stockout, sobrestock, cola y cash; 5. resolver softlocks; 6. revisar valores de clientes; 7. cerrar click-through y UI; 8. crear build externa; 9. actualizar QA/trazabilidad; 10. documentar known issues. # 111. Criterios económicos de H6 - Golden Path económico completo en build externa; - siete días sin S0/S1; - cash, stock, orders, transactions y ledger coherentes tras reload; - ningún ingreso/coste duplicado; - catálogo canónico identificado; - valores candidatos versionados; - resumen diario correcto y legible; - no softlock razonable; - Player.log sin errores bloqueantes; - evidencia y checksum archivados. # 112. Severidad de defectos económicos | Nivel | Ejemplos | | — | — | | S0 | save destruye economía o corrupción general | | S1 | duplica dinero/stock, venta parcial,

softlock, checkout doble | | S2 | margen/valor incorrecto no explotable, resumen confuso importante | | S3 | copy, formato o feedback menor | | S4 | mejora de tuning/telemetría |

Un balance poco divertido puede bloquear H6 aunque no sea un bug técnico, si impide el recorrido o produce insolvencia recurrente. Se registra como design/balance issue con severidad según impacto. # 113. Control de cambios de balance Cada cambio usa ID, fecha, hipótesis, parámetros anteriores/nuevos, build, seeds, métricas, resultado y decisión. Se prohíbe cambiar varios grupos sin posibilidad de atribución. Los catálogos reciben versión o hash. Si cambia save semantics, se crea ADR/migración. # 114. Evidencia requerida - tabla de parámetros y hashes; - resultado de tests; - logs de campaña por día; - capturas de UI y resumen; - CSV/JSON de métricas; - save antes/después; - lista de defectos; - decisión de balance; - build, commit y checksum; - notas cualitativas del tester. Una hoja de cálculo sin build asociada no es evidencia de integración. Una build sin parámetros registrados no es reproducible. # 115. Telemetría local y privacidad Para H6 basta instrumentación local exportable. No se necesita servicio remoto. Los datos no deben incluir información personal; usan IDs de sesión/seed y eventos económicos. Cualquier telemetría online futura requerirá consentimiento, política y diseño independiente. # 116. Empleados y salarios — visión futura V0.1 definió categorías salariales, contratación, formación, cansancio y despidos. Todo está diferido. Cuando se abra, salarios serán postings periódicos y capacidad laboral afectará throughput. No se añaden costes laborales al vertical slice actual ni se usan sus cifras para juzgar rentabilidad H6. # 117. Investigación y expansión — visión futura La visión histórica incluye costes, tiempos y requisitos para ecommerce, publishing, estudio y plataforma. Se conserva como escalera conceptual. Antes de abrir una etapa se necesita economía estable de tienda, reserva operativa, no deudas y métricas. Los valores históricos de decenas o cientos de miles de euros son placeholders de progresión, no targets vigentes. # 118. Ecommerce y logística — visión futura Ecommerce introduce pedidos online, packaging, shipping, devoluciones y demanda separada. No debe reutilizar sin más el pedido a supplier. Necesitará inventario comprometido, SLA, coste de envío, fraude/errores, UI y QA. La Phase 1 solo prepara conceptos de order y warehouse. # 119. Publishing, desarrollo interno y plataforma — visión futura Estas fases cambian el modelo económico de retail a proyectos, royalties, salarios, infraestructura y riesgo. La presente especificación no fija sus fórmulas. Solo exige que futuras monedas/ledgers se integren sin romper la tienda y que cada sistema tenga documento propio. # 120. Dificultad y accesibilidad Dificultad modifica presión económica declarada; accesibilidad modifica presentación y control, no castiga recompensas. El modo relajado puede ofrecer caja o costes favorables, pero no debe ocultar información. El modo exigente no puede depender de bugs, clicks más rápidos o texto menos legible. # 121. Antiexploits - no vender la misma reserva dos veces; - no recibir la misma caja dos veces; - no cancelar después de obtener stock sin coste; - no recargar para duplicar postings; - no retirar mueble para clonar producto; - no cambiar catálogo para alterar order pendiente; - no generar cash por valores negativos/overflow; - no desbordar

long/int; - no saltar cierre/autosave; - no usar pausa o UI para repetir input. La prevención se basa en IDs, preflight, checked arithmetic, snapshots y tests, no en ocultar acciones. # 122. Aleatoriedad y seeds La selección de perfiles y búsqueda es determinista cuando corresponde. Cualquier variación futura usa seed persistido o derivable para reproducción. No se resemilla al cargar para obtener resultados favorables salvo diseño explícito. La campaña registra seeds. # 123. Work packages recomendados | WP | Objetivo | Salida | Ventana | | — | — | — | | ECO-WP-01 | Unificar catálogos | elegir IDs, precios, costes y autoridad runtime | Sprint 17 | | ECO-WP-02 | Reconciliar caja y ledger | definir fuente autoritativa y test de reconstrucción | Sprint 17 | | ECO-WP-03 | Fijar capital y día | campaña comparativa de parámetros | Sprint 17 | | ECO-WP-04 | Balancear seis productos | mix de margen, caja, rotación y espacio | Sprint 17 | | ECO-WP-05 | Balancear suppliers/cajas | trade-off coste frente a lote | Sprint 17 | | ECO-WP-06 | Cerrar softlocks | escenarios de caja/capacidad/stock | Sprint 17 | | ECO-WP-07 | Instrumentación local | CSV por día y producto | Sprint 17 | | ECO-WP-08 | Campaña siete días | build externa y evidencias | H6 | | ECO-WP-09 | Resumen diario | etiquetas y reconciliación | H6 | | ECO-WP-10 | Migración de saves | mapa de IDs y valores históricos | antes de catálogo final | | ECO-WP-11 | Fault injection checkout | rollback cross-aggregate/ledger | Sprint 17 | | ECO-WP-12 | Registro de balance | versiones, hashes y decisiones | continuo |

ECO-WP-01 — Unificar catálogos

Resultado esperado. elegir IDs, precios, costes y autoridad runtime. **Ventana:** Sprint 17. El paquete debe partir de build y parámetros congelados, incluir criterio de aceptación, cambios acotados y rollback. No puede mezclar expansión de alcance con tuning del vertical slice. **Cierre.** Código/assets actualizados, tests, campaña relevante, métricas, defectos, decisión y enlaces en producción/trazabilidad. Si la hipótesis no mejora el resultado, se revierte y se conserva el aprendizaje; un cambio no se mantiene solo por el coste invertido. ## ECO-WP-02 — Reconciliar caja y ledger **Resultado esperado.** definir fuente autoritativa y test de reconstrucción. **Ventana:** Sprint 17. El paquete debe partir de build y parámetros congelados, incluir criterio de aceptación, cambios acotados y rollback. No puede mezclar expansión de alcance con tuning del vertical slice. **Cierre.** Código/assets actualizados, tests, campaña relevante, métricas, defectos, decisión y enlaces en producción/trazabilidad. Si la hipótesis no mejora el resultado, se revierte y se conserva el aprendizaje; un cambio no se mantiene solo por el coste invertido. ## ECO-WP-03 — Fijar capital y día **Resultado esperado.** campaña comparativa de parámetros. **Ventana:** Sprint 17. El paquete debe partir de build y parámetros congelados, incluir criterio de aceptación, cambios acotados y rollback. No puede mezclar expansión de alcance con tuning del vertical slice. **Cierre.** Código/assets actualizados, tests, campaña relevante, métricas, defectos, decisión y enlaces en producción/trazabilidad. Si la hipótesis no mejora el resultado, se revierte y se conserva el aprendizaje; un cambio no se mantiene solo por el coste invertido. ## ECO-WP-04 —

Balancear seis productos **Resultado esperado.** mix de margen, caja, rotación y espacio. **Ventana:** Sprint 17. El paquete debe partir de build y parámetros congelados, incluir criterio de aceptación, cambios acotados y rollback. No puede mezclar expansión de alcance con tuning del vertical slice. **Cierre.** Código/assets actualizados, tests, campaña relevante, métricas, defectos, decisión y enlaces en producción/trazabilidad. Si la hipótesis no mejora el resultado, se revierte y se conserva el aprendizaje; un cambio no se mantiene solo por el coste invertido. ## ECO-WP-05 — Balancear suppliers/cajas **Resultado esperado.** trade-off coste frente a lote. **Ventana:** Sprint 17. El paquete debe partir de build y parámetros congelados, incluir criterio de aceptación, cambios acotados y rollback. No puede mezclar expansión de alcance con tuning del vertical slice. **Cierre.** Código/assets actualizados, tests, campaña relevante, métricas, defectos, decisión y enlaces en producción/trazabilidad. Si la hipótesis no mejora el resultado, se revierte y se conserva el aprendizaje; un cambio no se mantiene solo por el coste invertido. ## ECO-WP-06 — Cerrar softlocks **Resultado esperado.** escenarios de caja/capacidad/stock. **Ventana:** Sprint 17. El paquete debe partir de build y parámetros congelados, incluir criterio de aceptación, cambios acotados y rollback. No puede mezclar expansión de alcance con tuning del vertical slice. **Cierre.** Código/assets actualizados, tests, campaña relevante, métricas, defectos, decisión y enlaces en producción/trazabilidad. Si la hipótesis no mejora el resultado, se revierte y se conserva el aprendizaje; un cambio no se mantiene solo por el coste invertido. ## ECO-WP-07 — Instrumentación local **Resultado esperado.** CSV por día y producto. **Ventana:** Sprint 17. El paquete debe partir de build y parámetros congelados, incluir criterio de aceptación, cambios acotados y rollback. No puede mezclar expansión de alcance con tuning del vertical slice. **Cierre.** Código/assets actualizados, tests, campaña relevante, métricas, defectos, decisión y enlaces en producción/trazabilidad. Si la hipótesis no mejora el resultado, se revierte y se conserva el aprendizaje; un cambio no se mantiene solo por el coste invertido. ## ECO-WP-08 — Campaña siete días **Resultado esperado.** build externa y evidencias. **Ventana:** H6. El paquete debe partir de build y parámetros congelados, incluir criterio de aceptación, cambios acotados y rollback. No puede mezclar expansión de alcance con tuning del vertical slice. **Cierre.** Código/assets actualizados, tests, campaña relevante, métricas, defectos, decisión y enlaces en producción/trazabilidad. Si la hipótesis no mejora el resultado, se revierte y se conserva el aprendizaje; un cambio no se mantiene solo por el coste invertido. ## ECO-WP-09 — Resumen diario **Resultado esperado.** etiquetas y reconciliación. **Ventana:** H6. El paquete debe partir de build y parámetros congelados, incluir criterio de aceptación, cambios acotados y rollback. No puede mezclar expansión de alcance con tuning del vertical slice. **Cierre.** Código/assets actualizados, tests, campaña relevante, métricas, defectos, decisión y enlaces en producción/trazabilidad. Si la hipótesis no mejora el resultado, se revierte y se conserva el aprendizaje; un cambio no se mantiene solo por el coste invertido. ## ECO-WP-10 — Migración de saves **Resultado esperado.** mapa de IDs y valores históricos. **Ventana:** antes de catálogo final. El paquete debe partir de build y parámetros congelados, incluir criterio de aceptación, cambios acotados y rollback. No

puede mezclar expansión de alcance con tuning del vertical slice. **Cierre.** Código/assets actualizados, tests, campaña relevante, métricas, defectos, decisión y enlaces en producción/trazabilidad. Si la hipótesis no mejora el resultado, se revierte y se conserva el aprendizaje; un cambio no se mantiene solo por el coste invertido. ## ECO-WP-11 — Fault injection checkout **Resultado esperado.** rollback cross-aggregate/ledger. **Ventana:** Sprint 17. El paquete debe partir de build y parámetros congelados, incluir criterio de aceptación, cambios acotados y rollback. No puede mezclar expansión de alcance con tuning del vertical slice. **Cierre.** Código/assets actualizados, tests, campaña relevante, métricas, defectos, decisión y enlaces en producción/trazabilidad. Si la hipótesis no mejora el resultado, se revierte y se conserva el aprendizaje; un cambio no se mantiene solo por el coste invertido. ## ECO-WP-12 — Registro de balance **Resultado esperado.** versiones, hashes y decisiones. **Ventana:** continuo. El paquete debe partir de build y parámetros congelados, incluir criterio de aceptación, cambios acotados y rollback. No puede mezclar expansión de alcance con tuning del vertical slice. **Cierre.** Código/assets actualizados, tests, campaña relevante, métricas, defectos, decisión y enlaces en producción/trazabilidad. Si la hipótesis no mejora el resultado, se revierte y se conserva el aprendizaje; un cambio no se mantiene solo por el coste invertido. # 124. Plantilla de experimento de balance

```
Experiment ID:
Hipótesis:
Build / commit / checksum:
Catálogo y hash:
Seed(s):
Parámetros control:
Parámetros variante:
Escenario y duración:
Métricas primarias:
Métricas de seguridad:
Resultados:
Observaciones cualitativas:
Defectos:
Decisión: ACCEPT / REJECT / ITERATE
Responsable y fecha:
```

125. Perfiles detallados de productos runtime

game-neon-drift — Neon Drift

Producto de tipo **juego físico**. Coste unitario 15.00 €, venta 15.00 €, margen unitario 0.00 €, margen bruto 0.00 %, markup 0.00 %, 12 unidades por caja y desembolso por caja 180.00 €. El valor es runtime Phase 1 y debe validarse con caja de 1.000.00 €. La revisión observa: barrera de compra, espacio, frecuencia de reposición, ventas por día, contribución a caja, stockout, capital inmovilizado y relación con otros SKU. El precio no se ajusta de forma aislada; se conserva

una razón de rol dentro del mix. Cualquier cambio actualiza catálogo, tests, save/migración si procede y evidencia de campaña. ## **case-cloud-runner** — Cloud Runner Case Producto de tipo **carátula/caja**. Coste unitario 8.00 €, venta 8.00 €, margen unitario 0.00 €, margen bruto 0.00 %, markup 0.00 %, 16 unidades por caja y desembolso por caja 128.00 €. El valor es runtime Phase 1 y debe validarse con caja de 1.000.00 €. La revisión observa: barrera de compra, espacio, frecuencia de reposición, ventas por día, contribución a caja, stockout, capital inmovilizado y relación con otros SKU. El precio no se ajusta de forma aislada; se conserva una razón de rol dentro del mix. Cualquier cambio actualiza catálogo, tests, save/migración si procede y evidencia de campaña. ## **console-vertex-one** — Vertex One Console Producto de tipo **consola**. Coste unitario 180.00 €, venta 180.00 €, margen unitario 0.00 €, margen bruto 0.00 %, markup 0.00 %, 2 unidades por caja y desembolso por caja 360.00 €. El valor es runtime Phase 1 y debe validarse con caja de 1.000.00 €. La revisión observa: barrera de compra, espacio, frecuencia de reposición, ventas por día, contribución a caja, stockout, capital inmovilizado y relación con otros SKU. El precio no se ajusta de forma aislada; se conserva una razón de rol dentro del mix. Cualquier cambio actualiza catálogo, tests, save/migración si procede y evidencia de campaña. ## **controller-orbit-pad** — Orbit Pad Controller Producto de tipo **mando**. Coste unitario 30.00 €, venta 30.00 €, margen unitario 0.00 €, margen bruto 0.00 %, markup 0.00 %, 6 unidades por caja y desembolso por caja 180.00 €. El valor es runtime Phase 1 y debe validarse con caja de 1.000.00 €. La revisión observa: barrera de compra, espacio, frecuencia de reposición, ventas por día, contribución a caja, stockout, capital inmovilizado y relación con otros SKU. El precio no se ajusta de forma aislada; se conserva una razón de rol dentro del mix. Cualquier cambio actualiza catálogo, tests, save/migración si procede y evidencia de campaña. ## **headset-signal-pro** — Signal Pro Headset Producto de tipo **auriculares**. Coste unitario 45.00 €, venta 45.00 €, margen unitario 0.00 €, margen bruto 0.00 %, markup 0.00 %, 4 unidades por caja y desembolso por caja 180.00 €. El valor es runtime Phase 1 y debe validarse con caja de 1.000.00 €. La revisión observa: barrera de compra, espacio, frecuencia de reposición, ventas por día, contribución a caja, stockout, capital inmovilizado y relación con otros SKU. El precio no se ajusta de forma aislada; se conserva una razón de rol dentro del mix. Cualquier cambio actualiza catálogo, tests, save/migración si procede y evidencia de campaña. ## **accessory-memory-core** — Memory Core Accessory Producto de tipo **accesorio**. Coste unitario 9.00 €, venta 9.00 €, margen unitario 0.00 €, margen bruto 0.00 %, markup 0.00 %, 10 unidades por caja y desembolso por caja 90.00 €. El valor es runtime Phase 1 y debe validarse con caja de 1.000.00 €. La revisión observa: barrera de compra, espacio, frecuencia de reposición, ventas por día, contribución a caja, stockout, capital inmovilizado y relación con otros SKU. El precio no se ajusta de forma aislada; se conserva una razón de rol dentro del mix. Cualquier cambio actualiza catálogo, tests, save/migración si procede y evidencia de campaña. # 126. Perfiles detallados de mobiliario runtime ## **checkout-counter** — Checkout Counter Huella 4×2 celdas, altura 1.1 m, capacidad 0, coste 450.00 €, interactivo **sí**, comprable **sí** y soporte de producto

no. Material `furniture-checkout` y ruta `Furniture/CheckoutCounter`. La aprobación combina economía y espacio: coste por capacidad, accesibilidad, circulación, líneas de visión, frecuencia de uso y rol funcional. Un mueble no comprable se trata como infraestructura inicial y no debe aparecer en catálogo de compra. La capacidad cero es válida para checkout o decoración, no para un display vendido como almacenamiento. `## wall-shelf` — Wall Shelf Huella 4×1 celdas, altura 2.2 m, capacidad 24, coste 180.00 €, interactivo `sí`, comprable `sí` y soporte de producto `sí`. Material `furniture-wall-shelf` y ruta `Furniture/WallShelf`. La aprobación combina economía y espacio: coste por capacidad, accesibilidad, circulación, líneas de visión, frecuencia de uso y rol funcional. Un mueble no comprable se trata como infraestructura inicial y no debe aparecer en catálogo de compra. La capacidad cero es válida para checkout o decoración, no para un display vendido como almacenamiento. `## central-shelf` — Central Shelf Huella 4×2 celdas, altura 1.6 m, capacidad 32, coste 260.00 €, interactivo `sí`, comprable `sí` y soporte de producto `sí`. Material `furniture-central-shelf` y ruta `Furniture/CentralShelf`. La aprobación combina economía y espacio: coste por capacidad, accesibilidad, circulación, líneas de visión, frecuencia de uso y rol funcional. Un mueble no comprable se trata como infraestructura inicial y no debe aparecer en catálogo de compra. La capacidad cero es válida para checkout o decoración, no para un display vendido como almacenamiento. `## low-display` — Low Display Huella 3×2 celdas, altura 0.9 m, capacidad 12, coste 160.00 €, interactivo `sí`, comprable `sí` y soporte de producto `sí`. Material `furniture-low-display` y ruta `Furniture/LowDisplay`. La aprobación combina economía y espacio: coste por capacidad, accesibilidad, circulación, líneas de visión, frecuencia de uso y rol funcional. Un mueble no comprable se trata como infraestructura inicial y no debe aparecer en catálogo de compra. La capacidad cero es válida para checkout o decoración, no para un display vendido como almacenamiento. `## featured-display` — Featured Display Huella 2×2 celdas, altura 1.1 m, capacidad 8, coste 240.00 €, interactivo `sí`, comprable `sí` y soporte de producto `sí`. Material `furniture-featured` y ruta `Furniture/FeaturedDisplay`. La aprobación combina economía y espacio: coste por capacidad, accesibilidad, circulación, líneas de visión, frecuencia de uso y rol funcional. Un mueble no comprable se trata como infraestructura inicial y no debe aparecer en catálogo de compra. La capacidad cero es válida para checkout o decoración, no para un display vendido como almacenamiento. `## backroom-storage` — Backroom Storage Huella 5×2 celdas, altura 2.4 m, capacidad 80, coste 220.00 €, interactivo `sí`, comprable `no` y soporte de producto `no`. Material `furniture-storage` y ruta `Furniture/BackroomStorage`. La aprobación combina economía y espacio: coste por capacidad, accesibilidad, circulación, líneas de visión, frecuencia de uso y rol funcional. Un mueble no comprable se trata como infraestructura inicial y no debe aparecer en catálogo de compra. La capacidad cero es válida para checkout o decoración, no para un display vendido como almacenamiento. `## receiving-crate` — Receiving Crate Huella 2×2 celdas, altura 0.8 m, capacidad 24, coste 35.00 €, interactivo `sí`, comprable `no` y soporte de producto `no`. Material `furniture-crate` y ruta

Furniture/ReceivingCrate. La aprobación combina economía y espacio: coste por capacidad, accesibilidad, circulación, líneas de visión, frecuencia de uso y rol funcional. Un mueble no comprable se trata como infraestructura inicial y no debe aparecer en catálogo de compra. La capacidad cero es válida para checkout o decoración, no para un display vendido como almacenamiento. **## decoration-plant** — Decorative Plant Huella 1×1 celdas, altura 1.2 m, capacidad 0, coste 25.00 €, interactivo **no**, comprable **no** y soporte de producto **no**. Material **decoration** y ruta **Furniture/DecorationPlant**. La aprobación combina economía y espacio: coste por capacidad, accesibilidad, circulación, líneas de visión, frecuencia de uso y rol funcional. Un mueble no comprable se trata como infraestructura inicial y no debe aparecer en catálogo de compra. La capacidad cero es válida para checkout o decoración, no para un display vendido como almacenamiento. **# 127.** Índice de decisiones ADR económicas **## ADR-0024** — Stack and Unit-Capacity Model **Fuente:** **live__ADR-0024_Stack_And_Unit_Capacity_Model.md.** **SHA-256:** **a621a76517cf2dce58d2f2a6444486853bceb8107e45e2ce0cead9da79b99f87.** The baseline leaves stack versus slot representation and capacity semantics open. Sprint 6 needs a deterministic model that does not assume shelf facings, physical volume or later economy data. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. **## ADR-0025** — Atomic Inventory Transfers **Fuente:** **live__ADR-0025_Atomic_Inventory_Transfers.md.** **SHA-256:** **4286c837998785523afe7b5818a07a7f09b63b8f94175a602f7133ae7c3df6ad.** A transfer touches two mutable containers. Sequential public mutations could remove from the source before discovering that the destination cannot accept the units. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. **## ADR-0026** — Sprint 6 Version and Persistence Boundary **Fuente:** **live__ADR-0026_Sprint_06_Version_And_Persistence_Boundary.md.** **SHA-256:** **88e809b7e5a461f57c5d9f943adaf6eeb0445f62ba3cd08440f5d2e45c48ed1f.** - Freeze target application version at 0.0.7. - Do not change SaveRootV1, schema versions or persistence DTOs. - Do not add scene or presentation integration. - Expose inventory collections as immutable snapshots only. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. **## ADR-0027** — Product and Supplier Authoring with ScriptableObjects **Fuente:** **live__ADR-0027_Product_and_Supplier_Authoring.md.** **SHA-256:** **86a7e5e37add2c9d6fe855fef6deda2f4ddfb8f00cdb1062dfdbec3eb0dcb13.** Sprint 6 introduced pure C# product definitions and inventory behavior. Sprint 7 requires concrete products and supplier catalogs that a designer can inspect and edit in Unity without coupling the domain model to **UnityEngine**. Esta

decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0028 — Purchase Orders Use Whole Shipment Boxes **Fuente:** `live__ADR-0028_Box_Based_Ordering.md`. **SHA-256:** `57af021d5e1412eadd90d34cdb62da418efa8d0bd0001d8da4c219bb9fa98f40`. Supplier catalogs need deterministic order, delivery and receipt quantities. Ordering arbitrary loose units would require packaging and partial-box rules that are not part of the current Vertical Slice scope. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0029 — Shipment-Box Receiving Is Atomic **Fuente:** `live__ADR-0029_Atomic_Box_Receiving.md`. **SHA-256:** `bf3785f3178e96118326edaf1a4134d4300f5817a838cb24bacdd7dc6884af0d`. The receiving flow crosses order state, delivery state and Sprint 6 inventory. A failure after partial mutation could duplicate or lose units. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0030 — Single-product display assignment **Fuente:** `live__ADR-0030_Single_Product_Display_Assignment.md`. **SHA-256:** `ff34c07064a44ae5ffb3db2f205b4ef1fe2fd330bed57702a84f53e4f428e4a5`. Each `DisplayInstance` has zero or one assigned product. Assignment must resolve through the product registry and satisfy optional category restrictions. A different product can be selected only after all stock is returned and the previous assignment is cleared. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0031 — Display capacity and visible units **Fuente:** `live__ADR-0031_Display_Capacity_and_Visible_Units.md`. **SHA-256:** `55398e2d4b0f0989db0716a124fd96d597cfe878d561d2546f8114c74009a900`. Capacity is measured in total logical units. `VisibleUnitLimit` is separate and cannot exceed capacity. Visible units are derived as `min(stock, visible limit)`. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0032 — Atomic manual restocking **Fuente:** `live__ADR-0032_Atomic_Manual_Restocking.md`. **SHA-256:** `3995a709fcd6235ec918570685e381fea53ef9620adb0bf10a0f25d2f0e6b970`. Restocking validates source type, assignment, product, quantity, stock and capacity before using the Sprint 6 transfer service. Sources are limited to Storage and Transit. Returns use the same transfer foundation in reverse. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación

explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0034 — Selección determinista y cola de spawn **Fuente:** `live__ADR-0034-Deterministic_Profile_Selection_and_Spawn_Queue.md`. **SHA-256:** `b3190da9985256d7704754263838fb9845526aaf470eec40c3e6501ffa6b8d0b`.

La selección usa pesos enteros y un roll explícito. La cola es FIFO, rechaza IDs de solicitud o instancia duplicados y solo se consume cuando el spawn se confirma. Esto permite tests reproducibles y evita aleatoriedad oculta. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0035 — Ciclo de vida y paciencia **Fuente:** `live__ADR-0035-Customer_Lifecycle_and_Patience.md`. **SHA-256:** `abae8d2eebdfc7d8b65386653fb1cbb78c351049ef3d838a6d8b0668dd1e1173`.

El estado mínimo es `WaitingToEnter` → `Entering` → `Browsing` → `Leaving` → `Despawned`. La paciencia se consume únicamente durante `Browsing`; al llegar a cero el cliente cambia a `Leaving` y su objetivo activo pasa a `Exit`. Sprint 9 no interpreta la paciencia como abandono de carrito porque el carrito pertenece a Sprint 10. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0038 — Reservation-backed cart **Fuente:** `live__ADR-0038-ReservationBackedCart.md`. **SHA-256:** `b75acced6289b8579e95ccb0e1dbe76dcaa3fafbbd9e4c16c8bda84aaa7a37fc`.

El carrito referencia reservas activas y no duplica inventario. El display mantiene stock físico hasta checkout. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0039 — Deterministic shopping search **Fuente:** `live__ADR-0039-DeterministicShoppingSearch.md`. **SHA-256:** `f68af75f570d688595409b8c6be5689989143acb084811e26f6e21f125bae81e`.

Los candidatos se ordenan por preferencia, producto y display. No se introduce azar ni evaluación económica. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0040 — Reservation provenance **Fuente:** `live__ADR-0040-ReservationProvenance.md`. **SHA-256:** `fb548c17172d591576640e2454ef14319057adc48b3dbd9944374ff306f3dee5`.

Cada reserva conserva cliente, display, producto, cantidad y estado. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0041 — Customer shopping session boundary **Fuente:** `live__ADR-0041-CustomerShoppingSessionBoundary.md`. **SHA-256:** `d6bf2284912c0e33992a0af84e445ad75fb6af270bb901ec0bf0f03942a11c08`.

CustomerShoppingSession se enlaza por CustomerInstanceId sin ampliar CustomerInstance. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0042 — Strict FIFO checkout queue **Fuente:** live__ADR-0042-StrictFifoCheckoutQueue.md. **SHA-256:** 094a7bdd2663b03a04f085dd8b9e0c775081f8b64eb6319858f1ac39b086df43.

Solo la primera entrada puede ser llamada, procesada o completada. La posición es 1-based y se recalcula después de completar o cancelar. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0043 — Single-entry checkout station **Fuente:** live__ADR-0043-SingleEntryCheckoutStation.md. **SHA-256:** 251308c9d85fe221f37152f5f9448a811db556b3b91816c20485e7a12949d5db.

Una estación solo procesa una entrada. La identidad se conserva durante Busy y debe coincidir al completar. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0044 — Preflight then commit checkout **Fuente:** live__ADR-0044-PreflightThenCommitCheckout.md. **SHA-256:** a58dfea3f71f24fd52ae8c7ab9d015aad8b31ad56c60e12162e4aff95783a864.

Todas las líneas se validan antes de mutar. El commit agrupa cantidades por display y ejecuta stock, carrito, reservas, sesión, cola, estación y transacción sin exponer estados intermedios. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0045 — Checkout idempotency **Fuente:** live__ADR-0045-CheckoutIdempotency.md. **SHA-256:** dcacaf08280bbc3fb2f7947d5d594d13d1d84c415dfa93e3c83148280a171d5d75.

El registro rechaza IDs duplicados y cualquier nuevo intento sobre un carrito ya completado. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0046 — Logical Store Day aggregate **Fuente:** live__ADR-0046-LogicalStoreDayAggregate.md. **SHA-256:** 4b0703008d657d9b20ef1554afcf827e8126540b156009d91cfc61c6270fcc7d.

StoreDay es el agregado autoritativo de estado y tiempo. Usa segundos enteros y no depende de MonoBehaviour, escenas o Time. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0047 — Closing admission gates **Fuente:** live__ADR-0047-ClosingAdmissionGates.md. **SHA-256:** 1bea6d3a71b2dda9f027398423d40013fed095318570d0c983c3a5ef5adbf2ef.

Closing deshabilita el spawner técnico y sella la cola de checkout. El sellado preserva entradas existentes y permite completar Processing. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0048 — Deterministic drain and resolution **Fuente:** `live__ADR-0048-DeterministicDrainAndResolution.md`. **SHA-256:** `930429172027b64eed649d79c17f55747622690c4355f303b4b1f572f2e53da`.

Los clientes activos son dirigidos a Exit. Las sesiones no procesadas se abandonan y liberan. El checkout en curso se conserva. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0049 — Closure readiness snapshot **Fuente:** `live__ADR-0049-ClosureReadinessSnapshot.md`. **SHA-256:** `8bbbed47b5de9fe5d2fb48f877e7bc1452a95aab891c3fcae14d6b06fea819df9`.

Closed exige cero clientes, cero cola, estación no Busy, cero reservas activas y cero sesiones pendientes. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0050 — Technical day summary **Fuente:** `live__ADR-0050-TechnicalDaySummary.md`. **SHA-256:** `523e7074e3ffda8a481a51a7fe1637494cf452b722c77823f268c3d7b773a1dc`.

El resumen registra actividad operacional sin precios, ingresos, impuestos ni ledger. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0051 — Integer minor-unit money **Fuente:** `live__ADR-0051-IntegerMinorUnitMoney.md`. **SHA-256:** `73384536261c238a733a2f818844f35d55f135835e6b5c97cdd7dae6d63ca795`.

Todo importe monetario usa `long` en unidades menores y un `Currency-Code` explícito. No se usa punto flotante. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0052 — Separate sale-price catalog **Fuente:** `live__ADR-0052-SeparateSalePriceCatalog.md`. **SHA-256:** `1a0a34e33c5a76ff1cfb80921aa4dbbdf223200b95c8e54bf52e95e1eece4018`.

`ProductDefinition` permanece sin economía. Los costes continúan en el catálogo de proveedor y los precios de venta viven en un catálogo económico separado. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0053 — Quote before physical checkout **Fuente:** `live__ADR-0053-QuoteBeforePhysicalCheckout.md`. **SHA-256:** `5c71902b6c89d51d471338f30be7fdf34014f33944ada9030b826b0149562d3a`.

El carrito se cotiza antes de llamar al CheckoutService existente. Un fallo de precio no muta inventario ni estado de cliente. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0054 — Idempotent economy ledger **Fuente:** live__ADR-0054-IdempotentEconomyLedger.md. **SHA-256:** f52b16bb87159d2616add98a3c44442ddfb65e0fe5e50fa4fea29fee33215246. El ledger es append-only y rechaza IDs duplicados y combinaciones repetidas de tipo + fuente. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0055 — Closed-day economic result **Fuente:** live__ADR-0055-ClosedDayEconomicResult.md. **SHA-256:** d0445e9824c68ff49cdc6836a4809e56bc768dfef1f504dd17f6207db76ef0af. El resultado diario solo se genera para días Closed y valida que los asientos de ingreso coincidan con CompletedCheckouts. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0056 — Compatible integrated save v2 **Fuente:** live__ADR-0056-CompatibleIntegratedSaveV2.md. **SHA-256:** 5cc539efb691df4398f23002613d08079fa8869bde84816f89a91be781ffcf6a. El schema v2 complementa el GameSessionSnapshot v1. No modifica la identidad de sesión, slot, día o efectivo del save skeleton. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0063 — Closed-day autosave idempotency **Fuente:** live__ADR-0063-ClosedDayAutosaveIdempotency.md. **SHA-256:** 8575a38534155149713d4c711e65216ce43402c06dce60d181fbad87d806ad59. El autosave solo acepta DayCycle Closed y registra el último DayId guardado por slot. Señales duplicadas no vuelven a escribir. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. ## ADR-0073 — Completed checkout snapshot records **Fuente:** live__ADR-0073-CompletedCheckoutSnapshotRecords.md. **SHA-256:** 7d401e745e557c63167dae48cd0f4b05b1d08f5c58a2e652b4321925fc35caa7. - customer Despawned; - shopping session CheckedOut; - reservation Consumed; - transaction Completed; - empty final queue; - available station; - checkout revenue ledger posting. Esta decisión se interpreta junto con sus tests y cierre de sprint. Si el catálogo o flujo Phase 1 la simplifica, la simplificación no elimina la decisión: debe existir adaptación explícita o deuda registrada. Una sustitución requiere nuevo ADR y referencia bidireccional. # 128. Inventario de fuentes y hashes El inventario registra 118 fuentes entre genealogía histórica

y paquete consolidado. Los hashes permiten repetir la auditoría y evitar que dos archivos con el mismo nombre se confundan. Los PDFs históricos existen en el paquete, pero se ha utilizado su Markdown fuente equivalente para extracción textual; los manifiestos y registros conservan la procedencia.

Fuente	Procedencia	Palabras	SHA-256	Uso
live__ADR-0024_Stack_And_Unit_Capacity_Model.md	operational	129	a621a76517cf2dce58d2f2a6444486853bceb8107e45e2ce0cead9da79b99f87	Genealogía / evidencia
live__ADR-0025_Atomic_Inventory_Transfers.md	operational	115	4286c837998785523afe7b5818a07a7f09b63b8f94175a602f7133ae7c3df6ad	Genealogía / evidencia
live__ADR-0026_Sprint_06_Version_And_Persistence_Boundary.md	operational	82	88e809b7e5a461f57c5d9f943adaf6eeb0445f62ba3cd08440f5d2e45c48ed1f	Genealogía / evidencia
live__ADR-0027_Product_and_Supplier_Authoring.md	operational	140	86a7e5e37add2c9d6fe855fef6deda2f4ddfbfa8f00cdb1062dfdbec3eb0dcbb13	Genealogía / evidencia
live__ADR-0028_Box_Based_Ordering.md	operational	130	57af021d5e1412eaddd90d34cbb62da418efa8d0bd0001d8da4c219bb9fa98f40	Genealogía / evidencia
live__ADR-0029_Atomic_Box_Receiving.md	operational	154	bf3785f3178e96118326edaf1a4134d4300f5817a838cb24bacdd7dc6884af0d	Genealogía / evidencia
live__ADR-0030_Single_Product_Display_Assignment.md	operational	59	ff34c07064a44ae5ffb3db2f205b4ef1fe2fd330bed57702a84f53e4f428e4a5	Genealogía / evidencia
live__ADR-0031_Display_Capacity_and_Visible_Units.md	operational	45	55398e2d4b0f0989db0716a124fd96d597cfe878d561d2546f8114c74009a900	Genealogía / evidencia
live__ADR-0032_Atomic_Manual_Restocking.md	operational	52	3995a709fcd6235ec918570685e381fea53ef9620adb0bf10a0f25d2f0e6b970	Genealogía / evidencia
live__ADR-0034_Deterministic_Profile_Selection_and_Spawn_Queue.md	operational	48	b3190da9985256d7704754263838fb9845526aaf470eec40c3e6501ffa6b8d0b	Genealogía / evidencia
live__ADR-0035_Customer_Lifecycle_and_Patience.md	operational	63	abae8d2eebdfc7d8b65386653fb1cbb78c351049ef3d838a6d8b0668dd1e1173	Genealogía / evidencia
live__ADR-0038-ReservationBackedCart.md	operational	23	b75acced6289b8579e95ccb0e1dbe76dcaa3fafbbd9e4c16c8bda84aaa7a37fc	Genealogía / evidencia
live__ADR-0039-DeterministicShoppingSearch.md	operational	24	f68af75f570d688595409b8c6be5689989143acb084811e26f6e21f125bae81e	Genealogía / evidencia
live__ADR-0040-ReservationProvenance.md	operational	16	fb548c17172d591576640e2454ef14319057adc48b3dbd9944374ff306f3dee5	Genealogía / evidencia
live__ADR-0041-CustomerShoppingSessionBoundary.md	operational	17	d6bf2284912c0e33992a0af84e445ad75fb6af270bb901ec0bf0f03942a11c08	Genealogía / evidencia
live__ADR-0042-StrictFifoCheckoutQueue.md	operational	31	094a7bdd2663b03a04f085dd8b9e0c775081f8b64eb6319858fac39b086df43	Genealogía / evidencia
live__ADR-0043-SingleEntryCheckoutStation.md	operational	25	251308c9d85fe221f37152f5f9448a811db556b3b91816c20485e7a12949d5db	Genealogía / evidencia
live__ADR-0044-PreflightThenCommitCheckout.md	operational	37	a58dfa3f71f24fd52ae8c7ab9d015aad8b31ad56c60e12162e4aff95783a864	Genealogía / evidencia
live__ADR-0045-CheckoutIdempotency.md	operational	21	dcaf08280bbc3fb2f7947d5d594d13d1d84c415dfa93e3c83148280a171d5d75	Genealogía / evidencia
live__ADR-0046-LogicalStoreDayAggregate.md	operational	29	4b0703008d657d9b20ef1554afcf827e8126540b156009d91cfc61c6270fcc7d	Genealogía / evidencia
live__ADR-0047-ClosingAdmissionGates.md	operational			

rational | 28 | 1bea6d3a71b2dda9f027398423d40013fed095318570d0c983c3a5ef5adbf2ef
 | Genealogía / evidencia | | live__ADR-0048-DeterministicDrainAndResolution.md
 | operational | 30 | 930429172027b64eed649d79c17f55747622690c4355f303b4b1f572f2e53da
 | Genealogía / evidencia | | live__ADR-0049-ClosureReadinessSnapshot.md |
 operational | 24 | 8bbbed47b5de9fe5d2fb48f877e7bc1452a95aab891c3fcae14d6b06fea819df9
 | Genealogía / evidencia | | live__ADR-0050-TechnicalDaySummary.md | opera-
 tional | 19 | 523e7074e3ffda8a481a51a7fe1637494cf452b722c77823f268c3d7b773a1dc
 | Genealogía / evidencia | | live__ADR-0051-IntegerMinorUnitMoney.md | ope-
 rational | 25 | 73384536261c238a733a2f818844f35d55f135835e6b5c97cdd7dae6d63ca795
 | Genealogía / evidencia | | live__ADR-0052-SeparateSalePriceCatalog.md |
 operational | 31 | 1a0a34e33c5a76ff1cfb80921aa4dbbdf223200b95c8e54bf52e95e1eece4018
 | Genealogía / evidencia | | live__ADR-0053-QuoteBeforePhysicalCheckout.md
 | operational | 30 | 5c71902b6c89d51d471338f30be7fdf34014f33944ada9030b826b0149562d3a
 | Genealogía / evidencia | | live__ADR-0054-IdempotentEconomyLedger.md |
 operational | 23 | f52b16bb87159d2616add98a3c44442ddfb65e0fe5e50fa4fea29fee33215246
 | Genealogía / evidencia | | live__ADR-0055-ClosedDayEconomicResult.md |
 operational | 27 | d0445e9824c68ff49cdc6836a4809e56bc768dfb1f504dd17f6207db76ef0af
 | Genealogía / evidencia | | live__ADR-0056-CompatibleIntegratedSaveV2.md |
 operational | 29 | 5cc539efb691df4398f23002613d08079fa8869bde84816f89a91be781ffcf6a
 | Genealogía / evidencia | | live__ADR-0063-ClosedDayAutosaveIdempotency.md
 | operational | 28 | 8575a38534155149713d4c711e65216ce43402c06dce60d181fbad87d806ad59
 | Genealogía / evidencia | | live__ADR-0073-CompletedCheckoutSnapshotRecords.md
 | operational | 51 | 7d401e745e557c63167dae48cd0f4b05b1d08f5c58a2e652b4321925fc35caa7
 | Genealogía / evidencia | | live__CURRENT_PROJECT_HANDOFF.md |
 operational | 160 | ea48666ee43dbf978bef364ee645cd3ff32f31e4e6dfbd96b9edfab316546028
 | Genealogía / evidencia | | live__S10_Acceptance_Matrix.md | operational |
 116 | 8a3558314d7d868e3e2ce0dc33fe62e7b1b33eddf1e70b2d6eab43c9b3600d8c
 | Genealogía / evidencia | | live__S10_Test_Plan.md | operational | 44
 | 55fc567ad4532a20b65b5c284c907e8a39d3498c598bee6533552780a5f10fa3 |
 Genealogía / evidencia | | live__S11_Acceptance_Matrix.md | operational |
 119 | a6f876fb02398be1ee466d2f9dde9613ad9c0fa2fc0088778b791ab9a3d00e8d
 | Genealogía / evidencia | | live__S11_Test_Plan.md | operational | 50
 | 75c9f46d3f588baa0e4942042264f3d5e62f609a38e975f604dce5645920b84e |
 Genealogía / evidencia | | live__S12_Acceptance_Matrix.md | operational |
 123 | 9b16fac6f71a8d05203d140a77a1b0c202a5f445556cc663208b04e5c45392b9
 | Genealogía / evidencia | | live__S12_Test_Plan.md | operational | 66
 | 30bd7ced538009bf712a71eb200d7ac3646cce58b3ee09f0d6cb339ff83055e9 |
 Genealogía / evidencia | | live__S13_Acceptance_Matrix.md | operational |
 125 | e7f134005949b363e84fadb28d740ff268adb3c42de47070fbcc968862319f2b
 | Genealogía / evidencia | | live__S13_Test_Plan.md | operational | 66
 | ae9f583cfca7e51d7ba01a00dc335de7a5dd31b86e1532321b6a1888d30ada25 |
 Genealogía / evidencia | | live__S14_Acceptance_Matrix.md | operational |
 136 | 6abc7f9f6b1f199f797ede8e4610be0819c89c52ba763b09489f76e5b6d2b3e9
 | Genealogía / evidencia | | live__S14_Test_Plan.md | operational | 75
 | a11b98e2c724cf7ba9b59a75906073d352a8365df289169934ee798329c63bb |
 Genealogía / evidencia | | live__S15_Acceptance_Matrix.md | operational |

141 | ef5c3d5a5d788d7a52965e858620ca76d260711f96e5132bb7938c765b6762da
| Genealogía / evidencia | | live__S15_Test_Plan.md | operational | 88
| d0d52d54433180bf3c4995a933def70b47ce5712a538d3abed7823f8a2dbc983 |
Genealogía / evidencia | | live__S6_Acceptance_Matrix.md | operational |
551 | b55fe039c6404ff22749ff07f20dc741ee277ece3a892d9d1a6a526eb61dab8e
| Genealogía / evidencia | | live__S6_Test_Plan.md | operational | 168 |
5201945d6beb104bfe2297c7686805a78c84ba67bd51e8d6f4472d0bb762a8b0 |
Genealogía / evidencia | | live__S7_Acceptance_Matrix.md | operational |
584 | e5517ab2f2b047aacb634c1ee7e42ea6f4b1b0f3bb6477cc7a0fb595439546b2
| Genealogía / evidencia | | live__S7_Test_Plan.md | operational | 242 |
| 0453ad65d4af260b590eb050e0d438566b48db60470c0cbfbad3110f224f6d8f |
Genealogía / evidencia | | live__S8_Acceptance_Matrix.md | operational |
330 | 514b68f2839e76b1757df9d2d457dca9701cbe8f3de4169c6e38fe7e7c1e576c
| Genealogía / evidencia | | live__S8_Test_Plan.md | operational | 103 |
| 700cbef3a75e066574ad7b1f0c97ae0a20be0bd2f4b6bb21012f83a665268b0f |
Genealogía / evidencia | | live__S9_Acceptance_Matrix.md | operational |
127 | 476e8342efea9323b73b46337890b795fe2b3bc500486fd08a9e9f5dbf6706ef
| Genealogía / evidencia | | live__S9_Test_Plan.md | operational | 87 |
b293d94d423091b2c31d627993d7b88b2f479dc55ef3cf638085c45abe0411ef | Ge-
nealogía / evidencia | | live__Sprint_06_Closure_Report.md | operational |
206 | b0997bd963c49340833d3d0e259c747f97b7ecd5574b3da5a77a8e51a4da0f60
| Genealogía / evidencia | | live__Sprint_06_Current_Status.md | operational |
184 | 393749de939a329f70e38cd7f6356f1f38f125c3a4a06495aed809813a55b401 |
Genealogía / evidencia | | live__Sprint_07_Closure_Report.md | operational |
278 | 00a4116e3af46415a56444f21cb913a1f5d7fe57daa4639f8dd053cecb8ae1 |
Genealogía / evidencia | | live__Sprint_07_Current_Status.md | operational |
216 | 28ea6d605bf2edcfd592207a4b99fae832913f988b2e6e14e2228296669b4583 |
Genealogía / evidencia | | live__Sprint_08_Closure_Report.md | operational |
334 | d6a2eee566596bde2f15c4cb2cb9fdd3ac0623bfcf6964714820d2ec1472d5f5 |
Genealogía / evidencia | | live__Sprint_08_Current_Status.md | operational |
214 | ac4bd550a7410946c583949ddc50a47efd32b921ad7b4f175568bedf7dd4101b
| Genealogía / evidencia | | live__Sprint_09_Closure_Report.md | operational |
446 | d683b02e85f86a4753e0e7fe4f84466c88cd702ffe6bacfb1743c21d11f61921 |
Genealogía / evidencia | | live__Sprint_09_Current_Status.md | operational |
88 | 1b5b8afdac50a3de23f1764f81d3ac749ad5701fa9408ab2286534bdd32cc984 |
Genealogía / evidencia | | live__Sprint_10_Closure_Report.md | operational |
470 | e9a44655c0ad3974168a876bc31ebcf4d71c1995eea797d8e38b0c1034ebabcc
| Genealogía / evidencia | | live__Sprint_10_Current_Status.md | operational |
89 | 893435f0a6ec143e27717287e3702e0d7306e9f5da085f3fcdab40994cbdb4b |
Genealogía / evidencia | | live__Sprint_11_Closure_Report.md | operational |
365 | b4ae4ddc090cdb00af473e64cbc19ac2285cdad28e1face724b2e7ac8e0b7725
| Genealogía / evidencia | | live__Sprint_11_Current_Status.md | operational |
91 | 824a6fd6c261f8cdf4c8ac7fa5b5454772248c28c63a006aff83581603b5e38c |
Genealogía / evidencia | | live__Sprint_12_Closure_Report.md | operational |
301 | e745ab6b920f0910b054d3c2c84b20db3ade4baef82e55a4560a3821febac57f
| Genealogía / evidencia | | live__Sprint_12_Current_Status.md | operational

| 71 | abf1580da3fab4d76250d9b370d441fc5f448f23257d745ccdf27f100769e682 |
 Genealogía / evidencia | | live__ Sprint_13_Closure_Report.md | operational
 | 509 | a8c1ad0958a4fc002aa2f13f47a7cfcf5b00d5a0f050a078fa78e96a9027d8fa |
 Genealogía / evidencia | | live__ Sprint_13_Current_Status.md | operational
 | 90 | 11fa2c9c81e37474d7bed83b95243be46a1a1015d81fd663df3cdaf55eea84e8 |
 Genealogía / evidencia | | live__ Sprint_14_Closure_Report.md | operational
 | 476 | 68527b99d210a091e21855b9d64de0cabcf8a8d650ae9772fbd7e68f259875edd |
 Genealogía / evidencia | | live__ Sprint_14_Current_Status.md | operational
 | 87 | 1635e3b630c1b7596d2d2c3dbcc68a574ec1e0de2494ddb1c1a0d2c286040bdd |
 Genealogía / evidencia | | live__ Sprint_15_Closure_Report.md | operational
 | 171 | 343d252b625e600cebcecc2fc9a37c825228dc7eab7052ace8ce353b5f4c859f |
 Genealogía / evidencia | | live__ Sprint_15_Current_Status.md | operational
 | 100 | 73b0bc0d6d52ab1ee422ae3768727e4084db01eddd16e79ec773cfeae71ccbb2 |
 Genealogía / evidencia | | v03__ Cartridge_And_Cloud_Economy_Balance_Specification_v0.1.md
 | baseline v0.3 | 4811 | a79f65cb5d960ff1e8fa6c83b8369b295547600f2f058163078f07a3b892c9e4
 Genealogía / evidencia | | v03__ Cartridge_And_Cloud_GDD_v0.4_PCSteam.md
 | baseline v0.3 | 17065 | 8939cf748467b18cee2e351b5eac309fe108b7fe02cd6da59a96230e9dd125fe
 Genealogía / evidencia | | v03__ Cartridge_And_Cloud_Initial_Content_Catalog_v0.1.md
 | baseline v0.3 | 3559 | 5b416cba7975a0ee42762eabb9ea236c30a19091fb3d3031285ac67ab0a83dcb
 Genealogía / evidencia | | v04__ Cartridge_And_Cloud_Economy_Balance_Specification_v0.1.md
 | baseline v0.4 | 4826 | f83578295bab346c04186abd7d5ca64c006565617f38e71b8c229605b717fab
 Genealogía / evidencia | | v04__ Cartridge_And_Cloud_GDD_v0.4_PCSteam.md
 | baseline v0.4 | 17080 | 88be82c3d3cfa4cb5965379e621b0aba522efa1217041cdc23a129ec4817485a
 Genealogía / evidencia | | v04__ Cartridge_And_Cloud_Initial_Content_Catalog_v0.1.md
 | baseline v0.4 | 3574 | 2bb04a31e3a2661e17e00f23ac0becc61b2c198a36fce87cf9f7aa1c76f07103
 Genealogía / evidencia | | v05__ 2__ CURRENT_PROJECT_HANDOFF.md
 | baseline v0.5 | 304 | 9c251d0f3c14584f5da35a50ab3a5b62564e6f1f656d903fa2590eda74e2e396
 Genealogía / evidencia | | v05__ CURRENT_PROJECT_HANDOFF.md |
 | baseline v0.5 | 304 | 9c251d0f3c14584f5da35a50ab3a5b62564e6f1f656d903fa2590eda74e2e396
 Genealogía / evidencia | | v05__ Cartridge_And_Cloud_Economy_Balance_Specification_v0.2.md
 | baseline v0.5 | 335 | 06c11a0e1ea6a93a2f3fd18794007ce2160fbbec1658d43456d4b16fecdd8c484
 Genealogía / evidencia | | v05__ Cartridge_And_Cloud_GDD_v0.5_PCSteam.md
 | baseline v0.5 | 538 | 17284e1e8d67332d5a8e76e56d194b7e176225dfb49e2729caf6664a96c6aa09
 Genealogía / evidencia | | v05__ Cartridge_And_Cloud_Initial_Content_Catalog_v0.2.md
 | baseline v0.5 | 287 | 02439c64569896a24e292ec42717612412828e784bf7c1c701fbc477d57b0d51
 Genealogía / evidencia | | v05__ Current_Project_Baseline_Record.md | base-
 line v0.5 | 49 | 518e52784ae6c8737a360e221905ab542dd0a7c54d164409b71208556d26f793
 Genealogía / evidencia | | v06__ 2__ CURRENT_PROJECT_HANDOFF.md
 | baseline v0.6 | 161 | 7a297d84a36ecdd9e4296a631fc5acbff882218d125e63312850e8f28b46cd87
 Genealogía / evidencia | | v06__ CURRENT_PROJECT_HANDOFF.md |
 | baseline v0.6 | 161 | 7a297d84a36ecdd9e4296a631fc5acbff882218d125e63312850e8f28b46cd87
 Genealogía / evidencia | | v06__ Cartridge_And_Cloud_Economy_Balance_Specification_v0.3.md
 | baseline v0.6 | 317 | 0f3a938d2dcfb83e03b20cc8b7902342d8124f42965cb9585c81a8854ce16abc
 Genealogía / evidencia | | v06__ Cartridge_And_Cloud_GDD_v0.6_PCSteam.md
 | baseline v0.6 | 484 | 333c7777584bcdee29c1caa8f90f4e43e3013e88cc339466a1a51f5bdd42c0d7
 Genealogía / evidencia | | v06__ Cartridge_And_Cloud_Initial_Content_Catalog_v0.3.md

| baseline v0.6 | 335 | 68f8841d7f5e83b1078bbf8c110e799165b74f5d75a823839d0acfd3086b333e
| Genealogía / evidencia | | v06__Current_Project_Baseline_Record.md | base-
line v0.6 | 73 | 98e04d98696a645237fcf80cbca55467a16a09e6bd0d46df1a74637a222ca2e4
| Genealogía / evidencia | | v06__Sprints_00_15_Consolidated_Closure_Ledger.md
| baseline v0.6 | 551 | 17c317dacaec6438f7a1180b6b2811184a0d995fb41964ed2fe9ab46e1d35aab
| Genealogía / evidencia | | 00_Enfoque_y_Alcance.md | consolidado | 18925 |
63dd6f0f1b9c2c80be2aec55718d18d9d031ce4acb14f677769d6e69ceaad21f | Auto-
ridad vigente / contexto | | 01_Game_Design_Document.md | consolidado |
19760 | a9cd9e3203abc5269c25c59dc7f626b0771df4ec2d65e781109ff146bceb2177
| Autoridad vigente / contexto | | 02_Vertical_Slice_Specification.md | consoli-
dado | 9834 | 75893f130116e88a08b8da5590dd81876a234581081eafaa8d08374c1a60513f
| Autoridad vigente / contexto | | 02_Vertical_Slice_Specification.preview.md |
consolidado | 9834 | 75893f130116e88a08b8da5590dd81876a234581081eafaa8d08374c1a60513f
| Autoridad vigente / contexto | | 03_Technical_Design_Document.draft.md |
consolidado | 13774 | 66264703c5ff4959d03093690e9845a98ec0e5025e685b9a639ac8cbb616cd12
| Autoridad vigente / contexto | | 03_Technical_Design_Document.md | conso-
lidado | 13774 | 66264703c5ff4959d03093690e9845a98ec0e5025e685b9a639ac8cbb616cd12
| Autoridad vigente / contexto | | 04_Modelo_de_Datos.draft.md | consolidado
| 12944 | d851a72b55742c2c704cc7c002929bc5894e7142511c6af843440bb193fb19d4
| Autoridad vigente / contexto | | 04_Modelo_de_Datos.md | consolidado |
12944 | d851a72b55742c2c704cc7c002929bc5894e7142511c6af843440bb193fb19d4
| Autoridad vigente / contexto | | 05_UX_Flow.md | consolidado | 8720 |
e08da5ff67fb073a8b950babe325ae58ddb2e121513dc2f553acd4f2c14b867e | Auto-
ridad vigente / contexto | | 06_Production_Roadmap_y_Sprint_Plan.md |
consolidado | 9418 | d00b156c0ad1c66069278119c55f76c738a577a3e2835f770208d1a5f2faadff
| Autoridad vigente / contexto | | 07_QA_Testing_Plan.md | consolidado |
12027 | bc9c05a3737e345546ef16e5390e034295ef35ad66c31647852aec81b4e7affe |
Autoridad vigente / contexto | | 08_QA_Testing_Matrix.xlsx | consolidado | bi-
nario | 233e57f01d2468fe13c136c8fcd3ee75b39bd3de6530349626ea98d4ff254214 |
Autoridad vigente / contexto | | 08_QA_Testing_Matrix_preview.png | conso-
lidado | binario | d0535417c725d7a9667f03ef1fe131af2d9f35b3b376e8749872d1e0c7d1cc01
| Autoridad vigente / contexto | | 09_CSharp_Coding_Standards.md | consoli-
dado | 12639 | 3b734d6cd49f31c09c10bb4941625e143f6c634e3fb50c157f0720e73b1c2a68
| Autoridad vigente / contexto | | 10_Unity_Project_Setup_Guide.md | conso-
lidado | 11361 | 31430bb0544edd9577323aa0009d6ca3df8b9fbcf6c8a150fc33df69b9f963a4
| Autoridad vigente / contexto | | 11_Build_y_Versioning_Guide.md | consoli-
dado | 12759 | 38a16a0f97505c0b405509dd3c7e3d1b50e77b89fbc3f4cb1931d3e04f602215
| Autoridad vigente / contexto | | 12_Excel_Maestro_de_Produccion.xlsx | bin-
solidado | binario | 405376cb64f49b34f1f842f3840c12654e9f08e2ec4534b5c149fa811e87dd83
| Autoridad vigente / contexto | | 12_Excel_Maestro_de_Produccion_preview.png
| consolidado | binario | 07de2e21fa1e568a006a7a48d21155dc18ef7d7b43dd2054361bc3efc9d3afb9
| Autoridad vigente / contexto | | 13_Trazabilidad_y_Control_de_Cambios.xlsx
| consolidado | binario | f674e21d1a3a1f0970a3d339f26f2a2b51a19b5c257c24b9d3fa4118a5cf50eb
| Autoridad vigente / contexto | | 13_Trazabilidad_y_Control_de_Cambios_preview.png
| consolidado | binario | 0cce6c1f535328a566c2de4846c11e95028da3ebce62dfe8eb48fd315f3caad9
| Autoridad vigente / contexto | | 14_Project_Binder_Indice_Maestro.md |

consolidado | 14030 | 0c24dcd47e0d44793e75e764e3fc2c146e5b1fea04d887d43feb0df558711559
 | Autoridad vigente / contexto | | 15_Guia_Maestra.md | consolidado | 18015 |
 879bc1925cccf77e10df6c104302f3afc72f71e8c58dd85ae4f06e84e6f8b624 | Autori-
 dad vigente / contexto | | 16_Auditoria_Global_de_Coherencia.md | consolida-
 do | 13508 | 4a62376b8aee623be40957ae0511d2dcab2919947f3abb2cef3502b9bb60863e
 | Autoridad vigente / contexto | | 17_Art_Bible.md | consolidado | 18971
 | 095c6cc42223a972c7faa68435595fc7c22f839fab62b2038371a0ef6c14239 |
 Autoridad vigente / contexto | | 18_Audio_Bible.md | consolidado | 20728
 | c2eabe93a7c26f70e1d4d77fb0ddcaa97e2cdd1e70247e637b9db5bd2f07ce92 |
 Autoridad vigente / contexto | | 19_UI_Style_Guide.md | consolidado | 20613
 | b34ca4db1eba7a536466b6a9a105d8d92396ade2108aa7dd99fa7d5d4eaeabc1a |
 Autoridad vigente / contexto |

129. Glosario, aceptación e historial

Término	Definición
Caja	saldo disponible persistido
Ingreso	posting generado por checkout confirmado
Coste de recepción	posting al recibir mercancía
Margen unitario	venta menos coste unitario
Margen bruto %	margen dividido por venta
Markup	margen dividido por coste
On-hand	unidades físicas en contenedor
Reservado	unidades comprometidas a cliente
Disponibile	on-hand menos reservado
Quote	valoración completa previa al commit
Posting key	clave idempotente tipo+fuentes
Resultado bruto técnico	ingresos de checkout menos costes de recepción
Tuning	ajuste de parámetros sin cambiar regla
Softlock	estado sin progreso ni recuperación razonable
Catálogo canónico	fuentes única de IDs y valores runtime

Criterios de aceptación del documento: genealogía completa; distinción entre valores históricos, técnicos y runtime; inventario de productos, muebles, suppliers, clientes y displays; fórmulas e invariantes; riesgos de doble catálogo; plan de tuning; escenarios; gates; ADR y fuentes con hash. El documento permanece **ACTIVE / NOT BASELINE-FROZEN** hasta completar la auditoría final del paquete. | Versión | Fecha | Cambio | Estado | | — | — | — | — | | 1.0 | 2026-07-01 | Consolidación de historia, implementación, valores, tuning y QA | ACTIVE

| | Próxima | tras Sprint 17 | Valores candidatos y resultados de campaña |
PENDING | | Baseline 1.0 | tras auditoría global | Correcciones y congelación |
NOT READY |

El siguiente artefacto especializado es `21_Initial_Content_Catalog.xlsx`, que debe convertir los catálogos históricos y actuales en un registro editorial único enlazado con esta especificación. La auditoría global se repetirá cuando finalice toda la generación documental. Referencia serializada vigente: `initialCashCents = 100000`, equivalente a 1.000,00 EUR. El literal se conserva para trazabilidad con el asset runtime. `##` Protocolo de gobierno de parámetros económicos

Los parámetros económicos no se modifican directamente durante una sesión de prueba sin dejar registro. Cada ajuste debe identificarse como corrección de defecto, tuning, cambio de contenido o cambio de regla. Una corrección de defecto restaura el comportamiento ya especificado; el tuning cambia un valor manteniendo intacta la regla; un cambio de contenido añade o retira un producto, proveedor, mueble o perfil; y un cambio de regla altera fórmulas, momentos de cobro, invariantes o condiciones de éxito. Las dos últimas categorías exigen impacto documental y, cuando afectan arquitectura o persistencia, ADR.

Todo cambio debe registrar valor anterior, valor propuesto, unidad, catálogo o archivo propietario, motivo, hipótesis, escenario de prueba, seed, versión, build, métricas observadas y decisión. No se acepta una justificación como “se siente mejor” sin describir qué fricción se pretendía corregir y qué evidencia confirmó o refutó la hipótesis. La valoración cualitativa sigue siendo necesaria, pero debe quedar acompañada de datos reproducibles.

Los ajustes se agrupan en lotes pequeños. Cambiar simultáneamente capital inicial, demanda, costes, precios, tiempos de entrega y capacidad impide atribuir el resultado. El lote ordinario debe modificar una familia coherente y conservar un control sin cambios. Los parámetros de seguridad —atomicidad, idempotencia, unidades enteras, conservación de inventario y no negatividad— no son variables de balance y nunca se relajan para obtener una curva más fácil.

La fuente canónica de un parámetro debe estar declarada. Mientras convivan `ContentCatalog.asset` y los catálogos técnicos, el registro debe indicar cuál gobierna la ejecución evaluada. Un valor duplicado no se considera sincronizado por coincidencia accidental. La campaña de tuning debe capturar los hashes de los catálogos y el commit para evitar comparar ejecuciones realizadas sobre datos distintos bajo la misma etiqueta.

Una propuesta se acepta cuando mejora la métrica objetivo sin degradar límites de seguridad, diversidad de estrategia, claridad de UX o capacidad de recuperación. Se rechaza cuando solo desplaza el problema, crea una estrategia dominante, introduce softlock o depende de un comportamiento no explicado al jugador. Se itera cuando la señal es prometedora pero la muestra o instrumentación no permite concluir. `##` Definition of Ready para un cambio de balance

Un cambio de balance está listo para implementarse cuando cumple todos los puntos siguientes:

1. Existe un problema observable, no solo una preferencia personal. El problema se describe mediante escenario, estado inicial, acción y resultado deseado.
2. Se identifica la fuente de verdad del valor: catálogo Phase 1, catálogo técnico, settings de clientes, settings de shopping, configuración de cola, mueble o código.
3. Se distingue si el cambio afecta precio, coste, capacidad, tiempo, probabilidad, demanda, comportamiento, presentación o persistencia.
4. Se enumeran sistemas dependientes. Un precio puede afectar checkout, ledger, UI, save, tests, tutorial y campaña; una capacidad puede afectar placement, navegación, reposición y clientes.
5. Existe una hipótesis mensurable. Ejemplo: reducir el coste por caja del producto A debería disminuir los bloqueos de liquidez en los días 1–2 sin elevar el beneficio acumulado de siete días por encima del rango objetivo.
6. Se define un control y al menos una variante. La comparación usa la misma seed y estado inicial cuando el sistema lo permite.
7. Se fijan métricas primarias y guardarraíles: caja mínima, stockout, ventas rechazadas, tiempo de cola, unidades inmovilizadas, beneficio bruto técnico y errores.
8. Se prepara rollback exacto, incluyendo valor anterior y archivo.
9. Se identifican casos límite: caja exactamente igual al coste, capacidad llena, stock reservado, pedido duplicado, recepción repetida y load posterior.
10. La tarea tiene criterio de cierre y evidencia requerida.

No está Ready un cambio que depende de un sistema futuro no autorizado, que mezcla migración de catálogos con rediseño completo de demanda, que carece de seed o build identificable, o que intenta resolver un defecto de lógica mediante valores extremos. Tampoco está Ready una petición de “hacerlo más difícil” sin definir para quién, en qué día, mediante qué presión y con qué vías de recuperación.

Para cambios de schema, IDs o dinero serializado, la Definition of Ready exige además estrategia de compatibilidad, migración, backup, prueba de corrupción y decisión sobre saves anteriores. Para cambios puramente editoriales en contenido, exige licencia, icono, presentación UI, categoría y compatibilidad con displays. ## Definition of Done para tuning y economía

Un cambio de balance está Done cuando la implementación y la validación están separadamente cerradas. La implementación requiere datos actualizados en la fuente canónica, ausencia de duplicados contradictorios, tests unitarios o de integración adaptados, y trazabilidad del commit. La validación requiere ejecución del escenario nominal, límites, Golden Path económico y, cuando la magnitud lo justifica, campaña de siete días.

La evidencia mínima incluye: build o entorno; commit; hashes de catálogos; seed;

estado inicial; tabla de parámetros antes/después; resultados; capturas o export de ledger; defectos abiertos; y decisión. Un cambio no se considera Done porque la prueba automatizada compile o porque una única partida resulte rentable.

Los invariantes deben permanecer en PASS: dinero entero; saldo no negativo salvo regla explícita; stock conservado; reservas no superiores a on-hand; checkout sin mutación parcial; postings idempotentes; pedidos recibidos una sola vez; y save/load sin duplicar transacciones. Cualquier violación convierte el resultado en defecto, no en observación de balance.

Debe demostrarse que la UI presenta el valor nuevo de forma coherente y que el jugador entiende coste, precio, stock y consecuencia antes de confirmar. Si cambia una caja de proveedor, debe revisarse el coste total, no solo el coste unitario. Si cambia un mueble, debe revisarse su relación entre coste, capacidad, huella y rol. Si cambia demanda, debe observarse diversidad de compra y no únicamente ingresos.

El cambio queda documentado en esta especificación o en el catálogo operativo correspondiente, y se registra en `13_Trazabilidad_y_Control_de_Cambios.xlsx`. Los valores anteriores no se borran sin historial. El rollback debe seguir siendo posible hasta que la candidata de hito quede aprobada. Tras aceptación, la campaña de referencia se actualiza y las comparaciones futuras usan la nueva baseline. ## Plan de migración entre `ContentCatalog.asset` y los catálogos técnicos

La coexistencia actual no debe resolverse copiando valores manualmente de un archivo a otro. Primero se define una taxonomía canónica de producto que incluya ID estable, nombre localizable, categoría, coste de adquisición, precio de venta, unidades por caja, proveedores permitidos, icono, prefab, tags de demanda y estado de disponibilidad. Los conceptos que solo existan en una capa deben quedar como campos opcionales o proyecciones, no perderse.

La migración propuesta se divide en seis pasos. **Paso 1: inventario.** Registrar todos los IDs Phase 1 y técnicos, sus referencias en saves, tests, escenas, catálogos, UI y código. **Paso 2: mapeo.** Determinar equivalencias reales, renombres, productos nuevos y productos históricos sin equivalente. `controller-orbit-pad` puede compartir identidad conceptual, pero no se asume equivalencia automática si precios, categorías o referencias difieren. **Paso 3: autoridad.** Elegir un catálogo canónico y definir adaptadores para servicios heredados durante la transición. **Paso 4: compatibilidad.** Crear tabla de alias o migración para IDs persistidos. **Paso 5: pruebas.** Validar creación de partida, load de saves anteriores, pedido, recepción, display, reserva, checkout y ledger. **Paso 6: retirada.** Eliminar la fuente duplicada solo cuando no queden referencias runtime y exista rollback.

La migración debe preservar dinero y unidades. Un producto renombrado no puede reaparecer como SKU adicional y duplicar stock. Las reservas y líneas de pedido deben resolverse al ID canónico. Los postings históricos conservan su

source ID o guardan un alias auditable; no se reescriben silenciosamente si eso destruye trazabilidad.

La decisión debe especificar dónde viven precios y costes. Es válido que el precio de venta pertenezca a una política o catálogo separado si existe una razón — por ejemplo, promociones—, pero la separación debe ser explícita. No se admite que Phase 1 y el dominio técnico calculen la misma venta con importes distintos según la ruta de UI.

El cierre exige una campaña de comparación con ambos flujos, pruebas de save/load y una auditoría de referencias. Hasta entonces, todo informe económico debe declarar qué capa produjo los datos. ## Playbook: liquidez baja y recepción bloqueada

Este escenario valida la decisión Phase 1 de cobrar mercancía al recibirla. Se prepara una partida con caja inferior al coste total de una entrega pendiente, almacén con capacidad y pedido listo. Se intenta recibir desde UI y desde cualquier ruta alternativa disponible. El resultado esperado es rechazo sin mutación parcial: caja, stock, estado del pedido, ledger y save permanecen coherentes. El mensaje debe explicar falta de fondos y coste requerido.

Después se eleva la caja exactamente al coste. La recepción debe confirmarse una sola vez, dejar saldo cero válido, añadir todas las unidades y crear un único posting de **SupplierReceivingCost**. Repetir la acción no debe volver a cobrar ni añadir stock. Guardar y cargar no debe reabrir la posibilidad de recepción.

Como prueba de balance, se observa si el jugador dispone de una vía razonable de recuperación antes de quedar bloqueado. Puede vender stock existente, cancelar o posponer una acción según reglas autorizadas, o seleccionar pedidos menores. Si el único estado posible exige dinero que no puede obtenerse, existe softlock. No se soluciona regalando dinero sin registro; se revisan capital inicial, tamaños de caja, coste por caja, orden de tutorial y capacidad de venta.

Las métricas son: número de intentos rechazados; caja mínima; valor del stock vendible; días hasta recuperación; pedidos pendientes; y claridad del feedback. Se comparan productos de alto desembolso como **console-vertex-one** con productos de caja más barata. El objetivo no es eliminar tensión de liquidez, sino hacerla legible, evitable y recuperable.

El criterio de cierre exige tests automáticos para atomicidad e idempotencia y recorrido manual en build. Cualquier diferencia entre el ledger y el saldo es A0/S0 económico. Un mensaje incorrecto o falta de desglose puede ser S1/S2 según impacto, pero no debe ocultarse como mero tuning. ## Playbook: sobrestock, capital inmovilizado y capacidad

Se inicia con capital suficiente para comprar varias cajas y se selecciona una combinación que llene almacén o displays. Se registra coste total, unidades, capacidad usada, ventas por día y caja restante. El escenario debe mostrar la diferencia entre un producto rentable por unidad y una compra perjudicial por volumen, rotación o desembolso.

La evaluación compara al menos tres estrategias: variedad equilibrada; concentración en el SKU de mayor margen porcentual; y concentración en el SKU de mayor margen absoluto. Se mantiene la misma seed de clientes cuando sea posible. Se observa si alguna estrategia domina sin riesgo, si la falta de variedad reduce demanda y si el jugador puede liquidar inventario sin promociones actualmente no implementadas.

Los muebles también forman parte del capital inmovilizado. **central-shelf**, **wall-shelf**, **featured-display** y **low-display** deben compararse por coste, capacidad, huella y visibilidad. No se debe concluir que un mueble es mejor solo por coste/capacidad si bloquea circulación o no soporta el tipo de producto. Infraestructura no comprable, como **backroom-storage**, no se suma como decisión de compra del jugador en la campaña actual.

Las métricas incluyen días de cobertura, porcentaje de capacidad, valor de stock a coste, ventas perdidas por stockout frente a caja inmovilizada, rotación por SKU, margen realizado y saldo mínimo. Se registra stock reservado separadamente para no interpretar compromiso de cliente como disponibilidad.

Un resultado sano ofrece decisiones con trade-offs: productos caros aportan margen pero tensionan caja; cajas grandes reducen frecuencia de pedido pero elevan inmovilización; displays eficientes cuestan espacio o visibilidad. Si una única compra inicial garantiza el éxito, el balance carece de profundidad. Si cualquier compra razonable causa quiebra, el onboarding es punitivo. El tuning se realiza sobre costes, cajas, demanda o capital de forma aislada y trazable. ## Playbook: saturación de cola y capacidad de checkout

La cola técnica tiene longitud máxima seis. El escenario fuerza llegadas y finalización de compra por encima de la capacidad de procesamiento. Debe diferenciarse entre cliente que aún compra, cliente que busca cola, cliente en cola, cliente atendido y cliente que abandona. Una venta solo existe después del commit de checkout.

Se registran tiempo medio y máximo de espera, ocupación de cola, abandonos, cesta media, ingresos potenciales perdidos y satisfacción. También se inspecciona si el jugador comprende la causa: falta de stock, checkout ocupado, cola llena o navegación. El audio y UI deben evitar contar varias veces un mismo evento económico.

La prueba funcional confirma que una cesta reservada mantiene consistencia mientras el cliente espera y que la liberación por abandono devuelve unidades según la regla. No debe venderse stock reservado a otro cliente si el modelo lo prohíbe, ni perderse tras una salida. El checkout ejecuta preflight sobre stock, reservas, total y estado del cliente antes de mutar.

Para balance, se compara la capacidad de cola con máximo de clientes activos seis, intervalo de llegada ocho segundos y duración operativa. Una combinación puede ser técnicamente válida pero producir saturación permanente o ninguna presión. El objetivo del vertical slice es generar momentos de priorización sin

convertir cada día en espera pasiva.

Si se cambia el intervalo de llegada o paciencia, se prueban perfiles por separado. El **impatient-visitor** no debe dominar todas las pérdidas; el **collector** no debe ocupar la cola indefinidamente. Las variaciones se evalúan con varias seeds. El cierre requiere ausencia de ventas duplicadas, reservas huérfanas, saldo incorrecto y clientes atascados, además de una distribución de espera aceptable. ## Playbook: campaña económica de siete días

La campaña de siete días no es una única ejecución informal. Se prepara una ficha por día con seed, caja inicial, stock por contenedor, pedidos pendientes, muebles disponibles, clientes atendidos, ventas, costes de recepción, saldo final y defectos. Se guarda al cierre y se carga al inicio siguiente para validar continuidad.

Día 1 comprueba onboarding y supervivencia: el jugador entiende pedido, recepción, display y primera venta sin quedar sin caja de forma irrecuperable. **Día 2** observa reposición y repetición; los pasos no deben depender de memoria externa. **Día 3** introduce presión de stock o variedad. **Día 4** comprueba que las decisiones anteriores tienen consecuencias visibles. **Día 5** busca cuellos de cola y disponibilidad. **Día 6** fuerza recuperación de un error razonable. **Día 7** valida cierre, resumen, persistencia y ausencia de deriva acumulativa.

La campaña registra ingresos de checkout y costes de recepción como resultado bruto técnico, pero no lo etiqueta como beneficio neto. Si no existen renta, salarios, impuestos o servicios, no se inventan en el informe. Los valores históricos de costes fijos sirven para fases futuras, no para corregir el resultado actual manualmente.

Las métricas mínimas por día son: caja mínima/máxima/final; ventas; unidades vendidas; pedidos recibidos; coste; margen realizado; stockout; abandonos; tiempo de cola; autosaves; y errores. A nivel campaña se calculan tendencia de caja, concentración de ventas por SKU, rotación, estrategia dominante y recuperación.

Se ejecutan al menos tres perfiles: conservador, equilibrado y agresivo. La campaña no exige que todos produzcan idéntico saldo, pero ninguno de los enfoques razonables debe depender de un exploit o caer en softlock inevitable. Cada ejecución conserva build, commit, hashes y evidencia. Un cambio de parámetros invalida la comparación directa salvo que se marque como nueva cohorte. ## Plantilla de ledger y conciliación diaria

La conciliación diaria compara tres vistas: saldo de sesión, suma de postings y efectos de inventario. El saldo final esperado se calcula como saldo inicial más ingresos de checkout menos costes de recepción y otros tipos explícitamente implementados. En el alcance actual, no se añaden costes históricos no conectados. Si aparece una diferencia, se investiga antes de continuar la campaña.

Campo	Descripción
Día / seed	identidad reproducible
Saldo inicial	valor cargado o creado
CheckoutRevenue	suma de postings únicos
SupplierReceivingCost	suma de postings únicos
Otros postings	solo tipos realmente implementados
Saldo esperado	cálculo independiente
Saldo observado	estado runtime/save
Diferencia	debe ser cero
Unidades recibidas	por SKU y pedido
Unidades vendidas	por SKU y checkout
Stock esperado	inicial + recibido - vendido - pérdidas autorizadas
Stock observado	contenedores + reservas según estado
Claves duplicadas	debe ser cero
Evidencia	log, export o captura

La conciliación se ejecuta antes y después de save/load. Se repiten recepción y checkout con la misma source ID para probar idempotencia. Un posting duplicado rechazado no debe modificar saldo. Una transacción fallida no debe dejar una entrada de ledger huérfana ni consumir stock.

Los informes de UI pueden mostrar resúmenes, pero la auditoría usa datos de dominio. La presentación monetaria convierte centavos a EUR sin perder precisión y aplica formato localizable. No se compara texto renderizado con `float`; se compara el valor entero y después la representación.

Cuando se introduzcan nuevos tipos —renta, salarios, impuestos, servicios, promociones—, cada uno requiere enum o ID estable, política de posting key, momento de reconocimiento, tests y actualización de la fórmula de resultado. Hasta entonces, el ledger actual es deliberadamente parcial y debe nombrarse como tal. ## Registro de riesgos económicos y respuesta

Riesgo	Señal	Impacto	Respuesta
doble catálogo	mismo concepto con ID/precio distinto	resultados no comparables	migración y autoridad explícita
softlock de caja	no existe acción que genere fondos	campana bloqueada	revisar cajas, capital y recuperación
estrategia dominante	un SKU supera siempre a los demás	baja diversidad	ajustar demanda, riesgo y capital
sobrestock invisible	caja baja sin explicación	frustración	métricas y UI de coste/capacidad

Riesgo	Señal	Impacto	Respuesta
venta duplicada	dos postings por checkout	corrupción crítica	idempotencia y source IDs
recepción duplicada	stock/coste repetido	inflación o pérdida	estado final y clave única
stock negativo	reservas/ventas superan existencia	incoherencia	preflight y conservación
resultado mal nombrado	bruto llamado neto	decisiones erróneas	taxonomía contable precisa
tuning no reproducible	no hay seed/hash/build	aprendizaje perdido	plantilla obligatoria
valores hardcoded	cambio requiere código	riesgo y divergencia	catálogo/configuración
histórico tratado como actual	se aplican costes no implementados	falsa documentación	etiquetar HIS-TORY/VISION
pago ambiguo	placement versus receipt	UX y ledger contradictorios	fijar momento canónico
inflación de métricas	se cuentan ventas intentadas	balance optimista	medir solo commits
campana insuficiente	una partida define el tuning	sesgo	varias seeds y estrategias

Cada riesgo tiene owner y fecha de revisión en producción. Los riesgos que afectan integridad se tratan antes que la diversión del tuning. No se continúa ajustando precios mientras existan duplicaciones, saldo incoherente o saves no deterministas. Los riesgos de presentación se priorizan si impiden entender la consecuencia económica, porque una regla correcta pero opaca produce decisiones injustas.

La mitigación debe ser verificable. “Vigilar” no es una acción suficiente; debe convertirse en test, métrica, alerta, validación de catálogo o gate. Los riesgos aceptados se documentan con alcance y fecha de reevaluación. ## Protocolo de rollback de parámetros y catálogos

Todo lote de tuning conserva una instantánea de los archivos modificados y un registro de valores. El rollback debe poder ejecutarse sin editar manualmente decenas de referencias. Para ScriptableObjects, se conserva GUID y se revierte contenido, evitando crear un asset nuevo que rompa referencias. Para IDs, se requiere migración inversa o alias mientras existan saves afectados.

Se activa rollback cuando aparece corrupción, incumplimiento de invariantes, regresión S0/S1, estrategia dominante severa, softlock nuevo, incompatibilidad de save o resultados imposibles de atribuir. También puede revertirse un lote válido técnicamente si no mejora la experiencia y aumenta complejidad.

El procedimiento es: detener nuevas ejecuciones; etiquetar build afectada; conservar evidencia; revertir commit o valores; limpiar únicamente datos de prueba autorizados; ejecutar tests de economía e inventario; crear partida nueva; cargar save compatible; repetir escenario causal; y registrar decisión. No se oculta el experimento fallido: queda como cambio rechazado con aprendizaje.

Si el rollback afecta un catálogo canónico ya publicado, se revisan pedidos, reservas, displays y ledger persistidos. Los postings históricos no se recalculan para coincidir con precios nuevos; representan transacciones realizadas bajo la versión anterior. El save debe conservar suficiente información para interpretar o mostrar el importe original.

Los cambios de balance no se aplican directamente sobre una candidata H6 sin una ventana de estabilización. Si surge una corrección tardía imprescindible, se limita el alcance, se repite la campaña afectada y se genera una nueva build con checksum. La candidata anterior permanece archivada y marcada como sustituida. ## Criterios de canonización para 21_Initial_Content_Catalog.xlsx

El siguiente catálogo editorial debe transformar los datos dispersos en un registro único, sin convertirse por sí solo en fuente runtime no versionada. Cada fila de producto debe incluir ID canónico, alias históricos, nombre, categoría, generación, rareza o tags, proveedor, coste, venta, unidades por caja, margen, markup, icono, prefab, estado, sprint, licencia y evidencia. Cada mueble debe incluir coste, huella, altura, capacidad, flags de interacción/compra, material, prefab y rol.

El Excel debe señalar la procedencia exacta de cada valor: `ContentCatalog.asset`, catálogo técnico, documento histórico, propuesta o placeholder. Los valores propuestos no se mezclan con actuales. Las fórmulas de margen usan centavos enteros como entradas y no sustituyen la lógica runtime. El libro puede detectar anomalías —venta menor que coste, cajas demasiado caras para capital inicial, IDs duplicados, alias ambiguos—, pero la autoridad final debe quedar definida por el pipeline de datos.

La canonización requiere un estado por fila: CURRENT-RUNTIME, TECHNICAL-LEGACY, HISTORICAL, PROPOSED, DEFERRED o RETIRED. Un mismo concepto puede tener varias filas históricas, pero solo una versión vigente por contexto. Las relaciones con suppliers y perfiles de cliente deben usar IDs, no nombres libres.

Antes de exportar a Unity o copiar valores, se valida que el catálogo no contenga marcas reales, licencias incompletas o assets ausentes. Los cambios se rastrean mediante Change ID y versión. El hash del libro forma parte de la campaña de balance.

La aceptación de 21 exigirá reconciliar los seis productos Phase 1 con los seis técnicos, documentar no equivalencias, definir alias y elegir el camino de migración. Hasta ese momento, esta especificación seguirá marcando el doble catálogo como deuda A1/A2 según el flujo afectado.